

CAMPUSLY — The Campus Economy Platform

Complete Startup Blueprint & Business Plan

One platform. Every need on campus. Campusly is a trusted, closed-ecosystem super-app built for university campuses — starting at SRM Institute of Science and Technology — where students buy, sell, earn, collaborate, eat, and connect, all within a verified student community.

Table of Contents

1. [The Problem](#)
2. [The Solution](#)
3. [Product — Core Modules](#)
4. [Why Now & Why SRM](#)
5. [Business Model & Revenue Streams](#)
6. [Go-To-Market Strategy](#)
7. [Competitive Landscape](#)
8. [Scaling Roadmap](#)
9. [Team Structure](#)
10. [Financial Projections](#)
11. [Risk Analysis & Mitigation](#)
12. [Vision & Endgame](#)

1. The Problem

University campuses are dense, high-activity ecosystems — thousands of students living, learning, and transacting in a compressed geography — yet they remain largely underserved by existing platforms. The friction points are significant and well-documented.

Trust is broken on open marketplaces. When a student at SRM tries to sell a graphics tablet or buy a second-hand textbook, they resort to informal WhatsApp groups, public Facebook Marketplace listings, or OLX — all of which lack identity verification, buyer protection, and any sense of accountability. Scams, no-shows, and disputes are common.

Skill exchange has no infrastructure. A student who can edit videos, debug code, or tutor calculus has no formal channel to offer these services to peers. Fiverr and Upwork are globally competitive and transactionally impersonal. Campus talent goes untapped and unmonetized.

Food and convenience delivery is inadequate. Third-party apps like Swiggy and Zomato either don't service campus interiors, have long delivery windows, or charge fees that make small orders uneconomical. Students who want food from the hostel canteen, a specific stall in the food court, or a quick errand run have no reliable option.

Micro-work has no formal channel. Students frequently need help — carrying luggage, picking up a parcel, waiting in a queue, printing documents — and other students are willing to do it for a small fee. This market is entirely informal today.

Events and team formation are fragmented. Hackathons, cultural fests, startup competitions, and project collaborations are announced across Instagram pages, notice boards, WhatsApp forwards, and college portals — with no single discovery layer and no tooling for team formation.

The net result is a campus economy that operates largely in the dark — inefficient, untrustworthy, and unmeasurable.

2. The Solution

Campusly is a verified, closed-loop campus super-app. Unlike general-purpose marketplaces, Campusly is gated to students and staff with institutional email verification, meaning every actor on the platform has a real identity tied to their university. This single design decision changes the entire trust dynamic.

The platform aggregates six core activity categories — marketplace, skills, errands, food, micro-jobs, and events — into one unified application with a shared wallet, unified reputation system, and campus-specific context. Students do not need to leave the app to accomplish any campus-related transaction.

The core insight is this: **the campus is the product**. The closed geography, shared social graph, and institutional accountability create network effects and trust that no open marketplace can replicate. Campusly doesn't compete with Amazon or Swiggy — it makes them irrelevant within campus boundaries.

3. Product — Core Modules

3.1 The Marketplace — Buy & Sell

Students list and purchase second-hand physical goods within the campus ecosystem. Every listing is tied to a verified student profile, and all transactions are processed through the in-app wallet with escrow protection — funds are released to the seller only after the buyer confirms receipt.

Core categories include textbooks and notes, electronics and gadgets, furniture and room accessories, clothing and accessories, sports equipment, and stationery. Listings are discoverable by hostel block proximity, department, or category filter. A "Campus Pick-Up" system maps exchange points to well-known campus landmarks for safe, convenient handoffs.

The platform charges a 5–8% transaction fee on completed sales. A premium listing feature — featured placement, photo enhancement, auto-boost — is available as a paid add-on.

3.2 SkillHub — Services & Collaboration

SkillHub is the campus freelance marketplace. Students create verified skill profiles — UI/UX design, video editing, web development, content writing, photography, music production, tutoring,

language coaching, and more — and offer these services to peers, student clubs, and eventually campus businesses.

Unlike Fiverr, SkillHub is not a global race to the bottom on pricing. Because both parties are campus-verified, there is social accountability. Reviews carry weight because the reviewer and reviewee may share classes, hostels, or friend circles.

Collaboration Boards allow students to post project requirements — "I'm building an app for my capstone, need a UI designer and a backend developer" — and invite applications. This doubles as a team-formation engine for academic and entrepreneurial projects.

3.3 RunIt — Errands & Convenience Tasks

RunIt connects students who need a task done with students willing to do it for a fee. Tasks can be as simple as picking up a parcel from the campus post office, dropping clothes at the laundry, getting a printout from the library, or buying a specific item from the campus store.

The requestor posts a task with a fixed budget; nearby RunIt Runners (verified students who have opted into the runner role) accept and complete it. The system uses real-time location to match tasks to the nearest available runner. RunIt is essentially Taskrabbit, built for and by students, at campus scale.

3.4 CampusEats — Food & Beverage Delivery

CampusEats enables students to order food from any campus eatery — canteens, food courts, mess counters, juice stalls — and have it delivered to their hostel room, classroom block, or any campus landmark by a student delivery partner.

The delivery network is student-operated. Any verified student with a bicycle or who is willing to walk routes can sign up as a delivery partner and earn per delivery. This creates a gig income stream for students while solving the last-mile food delivery problem on campus.

Restaurant-side, Campusly onboards campus food vendors at zero commission initially, building supply-side density before introducing a 10–15% take rate. This is the same playbook Zomato used in its early city-by-city expansion.

3.5 QuickGigs — Micro-Jobs & Task Board

QuickGigs is a board of small, time-bound paid jobs posted by students, faculty, student clubs, and campus organisations. Jobs include data entry, survey participation, event volunteering, social media management for a club, poster design for a fest, audience coordination at an event, and academic research assistance.

These are not long-term freelance engagements — they are one-off, low-commitment tasks that pay between ₹50 and ₹500 per job. The platform earns a 10% service fee. Faculty posting research-assistance tasks marks an important early institutional revenue line.

3.6 CampusConnect — Events, Teams & Community

CampusConnect is the discovery and social layer of the platform. It aggregates all campus events — fests, hackathons, workshops, sports matches, movie nights, placement prep sessions — in a

structured, filterable feed. Students can RSVP, buy event tickets, and register teams directly through the app.

A Team Finder feature allows students to post "looking for members" cards with their skills and project brief, enabling organic team formation for competitions, startups, and academic projects. This is particularly powerful in an engineering college context where cross-disciplinary collaboration is high-value but structurally difficult.

Community Boards allow hostel-wise, department-wise, and batch-wise groups to post notices, run polls, and coordinate — replacing the chaotic WhatsApp group dynamic with structured, searchable communication.

3.7 CampusWallet — Payments & Financial Layer

All transactions across every module flow through CampusWallet — an in-app wallet powered by a payment gateway integration (Razorpay / PhonePe Business). Students load money via UPI, debit card, or net banking and transact within the ecosystem.

Key wallet features include escrow-protected payments for marketplace and skill transactions, instant peer-to-peer transfers, earnings dashboard for sellers, runners, and delivery partners, and weekly payout cycles to bank accounts. The wallet data over time becomes a rich financial profile — laying the groundwork for future credit and BNPL (Buy Now Pay Later) products targeted at students.

3.8 Trust & Reputation System

Every user has a Campusly Score — a composite reputation metric built from transaction completion rate, buyer/seller reviews, response time, dispute history, and community contribution. High scores unlock premium features: priority task matching, verified seller badge, highlighted profile in SkillHub, and eventually, credit features.

A dispute resolution system handles conflicts with a structured evidence-submission and review process, with a campus-level moderation team (initially student volunteers, later paid staff) adjudicating cases.

4. Why Now & Why SRM

SRM Institute of Science and Technology is one of India's largest private universities, with over 52,000 students across its Kattankulathur campus alone. It has five hostels housing tens of thousands of students, a dense food court ecosystem, over 200 student clubs, and a thriving entrepreneurship culture with its own incubator (IIDCOE).

The campus has high smartphone penetration, UPI literacy, and a student body that is already transacting informally across WhatsApp and Instagram. The informal economy exists — Campusly simply formalises, digitises, and monetises it.

The timing is right for three reasons. First, the post-COVID generation of students is deeply comfortable with app-based commerce and gig-economy models. Second, Indian fintech infrastructure — UPI, Razorpay, ONDC — has matured to the point where building a campus wallet is low-friction. Third, no credible competitor has built a campus-specific closed-ecosystem product at scale in India. The space is wide open.

5. Business Model & Revenue Streams

Campusly operates a multi-sided platform model with diversified revenue streams, structured to be unit-economics positive at the campus level before scaling.

Transaction Commission is the primary revenue driver. The platform takes a percentage of every completed transaction across modules — 5–8% on marketplace sales, 10% on skill services, 10% on micro-jobs, and 10–15% on food orders. With thousands of daily micro-transactions at a campus of 50,000 students, this compounds quickly.

Premium Listings & Boosts allow sellers and skill providers to pay for enhanced visibility — featured placement on the home screen, priority algorithmic ranking, and highlighted search results. These are priced at ₹29–₹99 per listing or on a weekly subscription basis.

Campus Business Subscriptions are offered to campus vendors (canteens, stationary shops, print shops, laundry services) who want a managed storefront on Campusly. A monthly subscription of ₹500–₹2,000 gives them an in-app store, order management, and analytics.

Event Ticketing earns a 3–5% fee on every ticket sold through CampusConnect. For large fests, this is meaningful — SRM's Aaruush tech fest draws tens of thousands of attendees.

Sponsored Posts & In-App Advertising from brands targeting the student demographic — EdTech companies, laptop brands, stationery companies, food brands — can place contextually relevant ads within the feed. These are non-intrusive, clearly labeled, and priced on a CPM or fixed-period basis.

Data & Insights (B2B, Phase 2) — aggregated, anonymised campus commerce data is a valuable asset for FMCG brands, research institutions, and EdTech companies. A B2B insights dashboard is a Phase 2 revenue line.

Financial Products (Phase 3) — CampusWallet transaction history enables a micro-credit product: small, short-tenure loans of ₹500–₹5,000 for students with high Campusly Scores. Partnering with an NBFC or pursuing an NBFC license post-Series A makes this viable.

6. Go-To-Market Strategy

Phase 0 — Stealth & Foundation (Months 1–3)

The founding team builds the MVP while simultaneously recruiting 50 Campus Ambassadors from SRM's most active student communities — club presidents, hostel wing representatives, and entrepreneurship cell members. These ambassadors are offered early equity or significant platform credits, not cash, ensuring they are invested in the platform's success.

A waitlist campaign is run through Instagram and LinkedIn targeting SRM students. The hook is a "Founding Member" badge — a permanent status marker visible on their profile that signals early adoption. This leverages the social status dynamics of a university community.

Phase 1 — Launch at SRM Kattankulathur (Months 4–6)

The platform launches with a 72-hour "Campus Sale" event — a hyperlocal version of a Big Billion Day, where students list items and the platform guarantees transaction safety and a cashback incentive for first transactions. The launch event is physical and digital simultaneously, held at key campus gathering points.

Supply and demand are seeded in parallel. On the supply side, 200+ listings are pre-loaded by ambassadors. On the demand side, a ₹50 wallet credit is offered to every new user who completes their first transaction within the first week.

Phase 2 — Depth at SRM (Months 7–12)

Focus shifts to increasing transaction frequency and module adoption. A referral programme — "Invite a friend, both get ₹30 wallet credit" — drives organic growth. Weekly themed campaigns (hostel sale weekends, skill-swap Wednesdays, quick-gig Fridays) create habitual usage patterns.

The food delivery module launches in Month 8 after onboarding the top 10 campus food vendors. Student delivery partners are recruited with a ₹500 sign-up bonus and assured minimum earnings for the first month.

Phase 3 — Adjacent Campus Expansion (Months 13–24)

The SRM playbook is packaged as a replicable campus launch kit and deployed at 5 additional campuses in Tamil Nadu and Andhra Pradesh — VIT Vellore, Anna University, Amrita Coimbatore, BITS Pilani Hyderabad, and Manipal University Jaipur. Each campus launches with a locally recruited Ambassador team following the same Phase 0–1 playbook.

7. Competitive Landscape

No direct competitor exists in the Indian campus-specific super-app space. The closest analogues and their limitations are as follows.

OLX / Facebook Marketplace — open to all geographies, zero trust infrastructure, no campus-specific features, no food or errand modules. Campusly's trust moat is structural: you cannot replicate closed-campus verification on a general-purpose platform.

Swiggy / Zomato — do not service campus interiors, have high delivery fees for small orders, do not support small campus vendors, and have no peer-to-peer delivery model. Campus-specific food delivery with student delivery partners is a cost and experience advantage.

Internshala / LinkedIn — serve long-term employment and internship needs. QuickGigs and SkillHub target micro-transactions and short-term campus-specific engagements that these platforms structurally ignore.

Fiverr / Upwork — global platforms with high competition and no campus trust layer. A student offering tutoring services on Campusly is accountable to their peer community in a way that a Fiverr seller is not.

WhatsApp Groups — the de facto incumbent. Campusly wins by offering what WhatsApp cannot: search and discovery, payment infrastructure, dispute resolution, and a reputation system.

The competitive moat Campusly builds is the **closed-campus network effect**: the more verified students join and transact, the more trustworthy and valuable the platform becomes — and the more painful it is to leave.

8. Scaling Roadmap

Stage 1 — Campus (Year 1–2): SRM + 5 Campuses

The immediate focus is achieving deep product-market fit at SRM, optimising unit economics per campus, and establishing the Ambassador playbook as a repeatable GTM motion. Success metric: ₹50L monthly GMV at SRM within 12 months of launch.

Stage 2 — University Network (Year 2–4): 50 Campuses Across India

With a validated playbook and growing brand recognition in student communities, Campusly targets 50 campuses across India's top 200 engineering and management institutions. A dedicated Campus Success team manages onboarding, localisation (language, vendor mix, cultural events), and ambassador programme quality. A franchise-lite model is explored where strong local ambassadors become Campus Operators with revenue sharing.

At this stage, the platform crosses 1 million verified users — a threshold that makes Campusly a serious advertising platform for student-focused brands. The B2B insights and advertising revenue lines are formally launched.

Stage 3 — Corporate & Alumni Extension (Year 4–6)

The most natural expansion beyond universities is into corporate campuses and tech parks — ecosystems that share the same structural characteristics as university campuses: closed geography, known identities, high transaction density. Products are adapted for a corporate audience: corporate marketplace (sell office equipment, buy lunch from the cafeteria), skill exchange between employees, internal gig board for project-based work.

Alumni networks of partner universities are activated as a trust bridge — alumni can use a read-only version of CampusConnect to post job opportunities, mentorship offers, and resource listings targeted back to their alma mater.

Stage 4 — Financial Super-App for Young India (Year 6+)

Having built a financial transaction layer used by millions of young Indians through their most formative years, Campusly has a unique asset: a rich, longitudinal financial identity for a demographic that traditional banks have no data on. The endgame is a financial super-app — offering student credit, savings products, insurance, and eventually investment tools — built on top of the trust and transaction infrastructure already in place.

The model here is analogous to how Grab evolved from ride-hailing to a full financial services platform in Southeast Asia, using their existing user base and transaction data as the foundation.

9. Team Structure

Founding Team Requirements

The founding team needs to cover three functional areas: product & technology, business development & partnerships, and operations & community. A four-to-five-person founding team is ideal.

CEO / Co-founder — Product vision, fundraising, institutional partnerships, and strategic direction. Must have strong communication skills and credibility within the SRM student/alumni community.

CTO / Co-founder — Full-stack technical architecture, mobile development, and engineering team leadership. Experience with scalable backend systems is essential from day one.

Head of Growth / Co-founder — Campus ambassador programme management, GTM execution, partnerships with clubs and vendors, and user acquisition.

Head of Operations — Logistics for food delivery and errands, vendor onboarding, dispute resolution, and payment reconciliation.

Early Hires (Month 1–6)

The first three hires should be two junior engineers (mobile and backend) and one community manager embedded on the SRM campus. All early hires should be current SRM students or recent alumni — this is a deliberate cultural and strategic choice.

Advisory Board

The advisory board should include one person from a successful Indian consumer startup (Zepto, Dunzo, or similar), one faculty member or administrator from SRM for institutional credibility and access, one investor with EdTech or consumer marketplace experience, and one legal advisor familiar with RBI payment regulations and university compliance.

10. Financial Projections

These projections are based on conservative transaction volume assumptions at SRM with 50,000 students and a 10% initial activation rate growing to 60% by end of Year 2.

Year 1 — SRM Pilot

Metric	Q1	Q2	Q3	Q4
Active Users	500	2,000	5,000	10,000
Monthly GMV (₹)	2L	8L	20L	50L
Monthly Revenue (₹)	14K	56K	1.4L	3.5L
Burn Rate (₹/month)	3L	3.5L	4L	5L

Year 1 total revenue: ~₹20L. Year 1 total burn: ~₹45L. Year 1 funding requirement: ₹50–60L (seed/pre-seed round or bootstrapped with savings + friends & family).

Year 2 — 6 Campuses

Metric	Value
Total Users	80,000
Monthly GMV	₹2.5 Cr
Monthly Revenue	₹20L
Monthly Burn	₹18L
EBITDA	Near breakeven

Year 3 — 25 Campuses

Metric	Value
Total Users	4,00,000
Annual GMV	₹80 Cr
Annual Revenue	₹6.4 Cr
EBITDA Margin	15%

Series A target: \$3–5M at a \$15–20M valuation, based on GMV growth, multi-campus traction, and wallet engagement metrics.

11. Risk Analysis & Mitigation

University Administration Resistance. Institutions may resist a third-party commercial platform operating within campus boundaries. Mitigation: approach administrations as partners, not vendors. Offer revenue sharing, student employment data, and branding opportunities. SRM's own incubator (IIDCOE) is a natural entry point for institutional buy-in.

Trust & Safety Incidents. A high-profile fraud, harassment, or dispute case could damage the platform's reputation fatally in a tight-knit campus community. Mitigation: invest in moderation and dispute resolution from Day 1, enforce identity verification strictly, and implement a zero-tolerance policy with immediate account suspension for verified misconduct.

Delivery & Operations Reliability. Food delivery and errand services are operationally complex. A failure here (late food, lost parcels) creates highly visible negative word-of-mouth. Mitigation: start with a limited delivery zone, set conservative delivery time estimates, and over-communicate order status. Launch with a satisfaction guarantee for the first 3 months.

Monetisation Timing. Introducing fees too early kills adoption; introducing them too late delays path to revenue. Mitigation: the first 90 days are completely free for all participants. Fees are introduced gradually, starting with optional premium features before mandatory transaction commissions.

Platform Cloning by Competitors. Once the model is validated, a well-funded player (Swiggy, a VC-backed startup) could attempt to replicate it. Mitigation: the campus network effect is the moat — a replicator must rebuild trust, verification, ambassador relationships, and vendor onboarding from scratch at every campus. Speed and depth of campus penetration is the primary defensive strategy.

12. Vision & Endgame

The immediate mission is to build the most trusted campus ecosystem in India. The medium-term mission is to become the operating system for the student economy. The long-term vision is to be the financial and social infrastructure for the first decade of every young Indian's adult life.

A student who joins Campusly at 18 — buying textbooks, earning from RunIt deliveries, getting their first freelance payment through SkillHub — is building a financial identity and a trust score. By the time they graduate, Campusly knows more about their economic behaviour, skills, and social graph than any bank or employer does. That is the asset.

The name, for now, is **Campusly**. The category it will own, if executed with discipline and speed, is the **student economy of India**.

CAMPUSLY — Engineering & Software Implementation Plan

Industry-Grade Technical Blueprint

This document defines the complete software architecture, engineering standards, infrastructure design, and phased implementation plan for Campusly at production scale. It is written to the standard of a Series A-stage engineering organisation and is intended as both a build guide and a due-diligence reference.

Table of Contents

1. [Engineering Philosophy](#)
2. [System Architecture Overview](#)
3. [Tech Stack Decisions](#)
4. [Microservices Architecture — Detailed Breakdown](#)
5. [Database Design & Data Architecture](#)
6. [API Design Standards](#)
7. [Mobile Application Architecture](#)
8. [Real-Time Systems](#)
9. [Payment & Wallet System](#)
10. [Trust, Safety & Moderation System](#)
11. [Search & Discovery Engine](#)
12. [Notifications Infrastructure](#)

- 13. [Authentication & Identity](#)
- 14. [Security Architecture](#)
- 15. [DevOps, CI/CD & Infrastructure](#)
- 16. [Observability & Monitoring](#)
- 17. [Phased Implementation Roadmap](#)
- 18. [Team Structure & Hiring Plan](#)
- 19. [Cost Architecture](#)
- 20. [Disaster Recovery & Business Continuity](#)

1. Engineering Philosophy

Campusly is built on four non-negotiable engineering principles that govern every architectural decision.

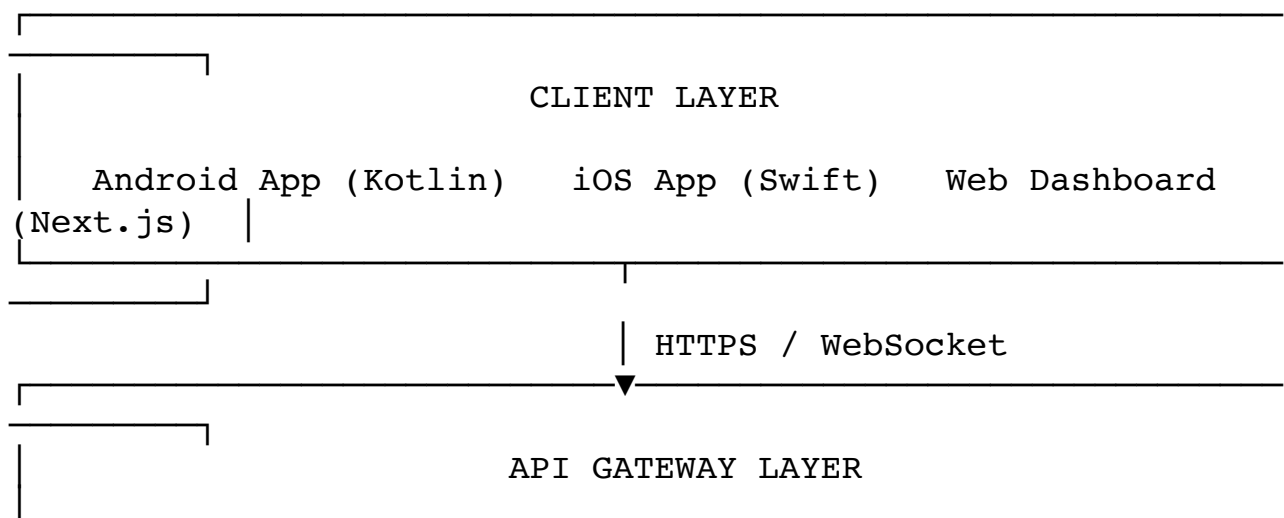
Start Monorepo, Scale to Services. The system begins as a well-structured monolith within a monorepo to maximise early development velocity. Service boundaries are identified and honoured in code from Day 1 — but deployment remains unified until scale demands otherwise. This avoids premature microservices complexity while preserving the ability to extract services cleanly when needed.

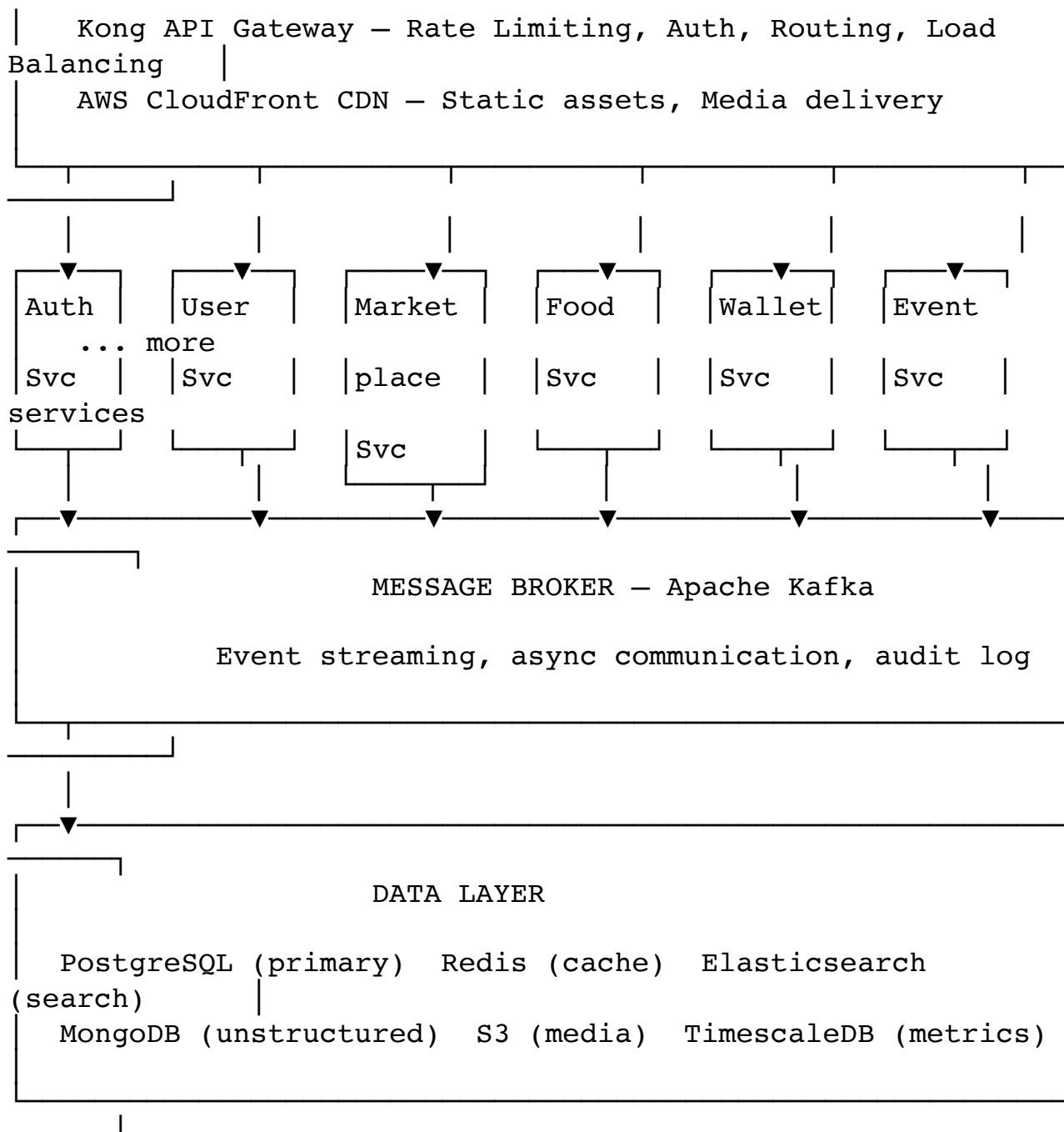
Mobile-First, API-First. Every piece of functionality is exposed via a versioned, documented REST or GraphQL API before any UI is built. The mobile application is a first-class consumer of the API — never a special case with privileged backend access.

Trust is a Feature. The reputation and identity layer is not an afterthought. It is designed as a core system with its own data model, event sourcing, and service from the beginning. Every user action that is trust-relevant emits a structured event that the Trust Service consumes.

Eventual Consistency is acceptable; Financial Inconsistency is not. The wallet and payment system operates under strict ACID guarantees with double-entry bookkeeping. All other systems can tolerate eventual consistency and are designed accordingly.

2. System Architecture Overview





The architecture is a **microservices system on Kubernetes**, with an API Gateway as the single entry point. Services communicate synchronously via REST/gRPC for request-response interactions and asynchronously via Kafka for event-driven workflows. The data layer is polyglot — different databases are chosen for different services based on their access patterns.

3. Tech Stack Decisions

Every technology choice is made against three criteria: proven production reliability, availability of engineering talent in India, and long-term community support.

Backend

Layer	Technology	Justification
-------	------------	---------------

Primary Language	Node.js (TypeScript)	Fastest development velocity, enormous npm ecosystem, excellent async I/O performance for API-heavy workloads
Secondary	Go	Used for high-throughput, low-latency services: matching engine, real-time location, wallet transactions
API Framework	NestJS (Node) / Gin (Go)	NestJS enforces clean architecture and DI patterns; Gin is minimal and performant for Go services
API Protocol	REST (primary) + gRPC (internal service-to-service)	REST for client-facing; gRPC for inter-service for performance
Message Queue	Apache Kafka	Industry-standard for event streaming, strong ordering guarantees
Task Queue	BullMQ (Redis-backed)	For background jobs, scheduled tasks, notification queues

Frontend / Mobile

Layer	Technology	Justification
Mobile Framework	React Native (initial) → Native (scale)	Single codebase for Android + iOS in Phase 1; migrate hot paths to native as scale demands
State Mgmt	Zustand + React Query	Lightweight, no boilerplate; React Query handles server sync
Web Framework	Next.js 14 (App Router)	Vendor dashboards, admin panel, campus analytics; SSR for SEO
Styling	NativeWind (mobile) + Tailwind CSS (web)	Consistent design token system across platforms

Data & Infrastructure

Layer	Technology	Justification
Primary Database	PostgreSQL 16	ACID compliance, strong consistency, excellent for relational data, JSONB for flexible schemas
Cache	Redis 7 (Cluster mode)	Session store, rate limiting, hot data cache, pub/sub for events
Search	Elasticsearch 8	Full-text search across listings, skills, events; geo-search
Document Store	MongoDB	Flexible schema for notifications, activity feeds, user profiles
Static Content	AWS S3 + CloudFront	Media (images, documents), static assets
Time-series	TimescaleDB	Platform analytics, GMV tracking, user activity metrics
CDN	AWS CloudFront	Global media delivery; essential for image-heavy
Container Orchestration	Docker + Kubernetes	Industry standard; AWS EKS for managed Kubernetes
Service Mesh	Istio	mTLS between services, traffic management, observability
Infrastructure as Code	Terraform	Reproducible infra, multi-environment support
Cloud Provider	AWS (primary) with Cloudflare for DNS/DDoS	Best-in-class managed services, strong India region (ap-south-1 Mumbai)

4. Microservices Architecture — Detailed Breakdown

Each service owns its data store, exposes a documented API, and communicates with other services only via API calls or Kafka events. There are no shared databases between services.

4.1 Auth Service

Responsibility: User registration, login, session management, OTP verification, institutional email verification, token issuance, and revocation.

Key Endpoints:

POST	/auth/register	– initiate registration, send OTP
POST	/auth/verify-otp	– verify institutional email OTP
POST	/auth/login	– password + device fingerprint login
POST	/auth/refresh	– refresh access token
POST	/auth/logout	– revoke refresh token
POST	/auth/forgot-password	– initiate reset flow

Key Design Decisions: Access tokens are short-lived JWTs (15 minutes). Refresh tokens are long-lived (30 days), stored in a Redis session store with device binding. Institutional email verification uses a campus-specific OTP sent to the @srmist.edu.in domain (and equivalents for other campuses). Future support for OAuth with university SSO systems (Shibboleth, CAS) is architected from the start via a pluggable identity provider interface.

Data Store: PostgreSQL (user credentials, sessions table), Redis (OTP cache, refresh token store).

4.2 User Service

Responsibility: User profiles, Campusly Score computation, preference management, social graph (follow/block), and user search.

Data Model (PostgreSQL):

```
users
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid()
  campus_id   UUID REFERENCES campuses(id)
  name        VARCHAR(100) NOT NULL
  username    VARCHAR(50) UNIQUE NOT NULL
  email       VARCHAR(255) UNIQUE NOT NULL
  phone       VARCHAR(15) UNIQUE
  department  VARCHAR(100)
  year_of_study SMALLINT
  hostel_block VARCHAR(20)
  bio         TEXT
  avatar_url  TEXT
  campusly_score DECIMAL(4,2) DEFAULT 5.00
  role        ENUM('student','staff','vendor','admin')
```

```

is_verified      BOOLEAN DEFAULT FALSE
created_at       TIMESTAMPTZ DEFAULT NOW()
updated_at       TIMESTAMPTZ DEFAULT NOW()

```

campus_score_events

```

id              UUID PRIMARY KEY
user_id         UUID REFERENCES users(id)
event_type      VARCHAR(50)    -- 'transaction_completed',
'review_received', etc.
delta           DECIMAL(4,2)   -- score change
metadata        JSONB
created_at      TIMESTAMPTZ DEFAULT NOW()

```

Campusly Score Algorithm:

The score is a weighted composite, recalculated nightly by a scheduled job:

```

Score = 0.35 × Transaction Completion Rate
      + 0.25 × Average Review Rating (normalised to 5)
      + 0.20 × Response Rate (messages replied within 2
hours)
      + 0.10 × Profile Completeness
      + 0.10 × Dispute Rate (inverse – fewer disputes =
higher score)

```

The score is event-sourced: every relevant user action emits a `score_event` to Kafka, consumed by the Score Worker, which maintains a running score and recalculates nightly.

4.3 Marketplace Service

Responsibility: Listing creation, management, search indexing, purchase flow, escrow triggers, and sold items management.

Data Model:

listings

```

id              UUID PRIMARY KEY
seller_id       UUID REFERENCES users(id)
campus_id       UUID REFERENCES campuses(id)
title           VARCHAR(200) NOT NULL
description      TEXT
price           DECIMAL(10,2) NOT NULL
category        ENUM('books', 'electronics', 'furniture', 'clothing', 'sports', 'other')
condition        ENUM('new', 'like_new', 'good', 'fair', 'poor')
images          TEXT[]          -- S3 URLs
pickup_point    VARCHAR(100)    -- campus landmark

```

```

status          ENUM('active','reserved','sold','removed')
boost_expires_at TIMESTAMPTZ -- premium listing expiry
created_at      TIMESTAMPTZ DEFAULT NOW()

```

listing_transactions

```

id              UUID PRIMARY KEY
listing_id      UUID REFERENCES listings(id)
buyer_id        UUID REFERENCES users(id)
seller_id       UUID REFERENCES users(id)
amount          DECIMAL(10,2)
platform_fee    DECIMAL(10,2)
escrow_id       UUID          -- reference to wallet escrow
status          ENUM('initiated','escrow_held','completed','disputed','refunded')
created_at      TIMESTAMPTZ DEFAULT NOW()
completed_at    TIMESTAMPTZ

```

Listing Flow:

User creates listing → Validation → Image upload to S3 → Elasticsearch index update → Kafka event(listing.created) → [Notification Service] → Feed Service updates nearby users

Purchase Flow:

Buyer initiates purchase → Wallet Service creates escrow → Listing marked 'reserved' → Physical exchange confirmed by buyer → Escrow released to seller → Review prompt triggered → Score events emitted for both parties

4.4 Skill Service (SkillHub)

Responsibility: Skill profile management, service listing, order management, collaboration board, and team formation.

Data Model:

skill_profiles

```

id              UUID PRIMARY KEY
user_id         UUID UNIQUE REFERENCES users(id)
headline        VARCHAR(200)
skills          TEXT[]
portfolio_urls  TEXT[]
hourly_rate     DECIMAL(8,2)
availability     JSONB    -- { mon: [9,18], tue: [9,18], ... }
rating          DECIMAL(3,2)

```

```

    total_jobs          INTEGER DEFAULT 0

skill_orders
  id                   UUID PRIMARY KEY
  client_id            UUID REFERENCES users(id)
  provider_id          UUID REFERENCES users(id)
  title                VARCHAR(200)
  description           TEXT
  agreed_price          DECIMAL(10,2)
  platform_fee          DECIMAL(10,2)
  escrow_id            UUID
  deliverables          TEXT[]
  deadline              DATE
  status
ENUM('pending','accepted','in_progress','delivered','complete
d','disputed')
  created_at           TIMESTAMPTZ DEFAULT NOW()

collaboration_posts
  id                   UUID PRIMARY KEY
  poster_id            UUID REFERENCES users(id)
  title                VARCHAR(200)
  description           TEXT
  required_skills       TEXT[]
  team_size             SMALLINT
  is_paid               BOOLEAN DEFAULT FALSE
  compensation          DECIMAL(10,2)
  deadline              DATE
  status               ENUM('open','filled','closed')
  applications          JSONB[] -- applicant references

```

4.5 Food Service (CampusEats)

Responsibility: Vendor management, menu management, order lifecycle, delivery partner assignment, and real-time order tracking.

Data Model:

```

vendors
  id                   UUID PRIMARY KEY
  campus_id            UUID
  name                 VARCHAR(150)
  location              POINT -- PostGIS geography
  landmark              VARCHAR(100)
  operating_hours       JSONB
  is_active             BOOLEAN DEFAULT TRUE

```

commission_rate DECIMAL(4,2) DEFAULT 0.12

menu_items

id	UUID PRIMARY KEY
vendor_id	UUID REFERENCES vendors(id)
name	VARCHAR(150)
description	TEXT
price	DECIMAL(8,2)
category	VARCHAR(50)
image_url	TEXT
is_available	BOOLEAN DEFAULT TRUE
prep_time_mins	SMALLINT

food_orders

id	UUID PRIMARY KEY
customer_id	UUID REFERENCES users(id)
vendor_id	UUID REFERENCES vendors(id)
delivery_partner_id	UUID REFERENCES users(id)
items	JSONB -- [{menu_item_id, qty, price}]
subtotal	DECIMAL(10,2)
delivery_fee	DECIMAL(6,2)
platform_fee	DECIMAL(6,2)
total	DECIMAL(10,2)
delivery_address	TEXT
status	

ENUM('placed', 'confirmed', 'preparing', 'ready', 'picked_up', 'delivered', 'cancelled')

estimated_delivery TIMESTAMPTZ

created_at TIMESTAMPTZ DEFAULT NOW()

Order State Machine:

placed → confirmed (vendor accepts) → preparing →

ready_for_pickup →

picked_up (delivery partner scans QR) → delivered (customer confirms) →

[payment released to vendor and delivery partner]

The delivery assignment uses a geospatial matching algorithm (described in Section 8 — Real-Time Systems) to identify the nearest available delivery partner.

4.6 Errand Service (RunIt)

Responsibility: Task posting, runner assignment, real-time tracking, completion verification, and payment release.

Data Model:

```

errand_tasks
  id                UUID PRIMARY KEY
  requester_id      UUID REFERENCES users(id)
  runner_id         UUID REFERENCES users(id)
  title             VARCHAR(200)
  description        TEXT
  budget            DECIMAL(8,2)
  platform_fee      DECIMAL(6,2)
  pickup_location   TEXT
  dropoff_location  TEXT
  status
ENUM('open','assigned','in_progress','completed','cancelled',
'disputed')
  created_at        TIMESTAMPTZ DEFAULT NOW()
  completed_at      TIMESTAMPTZ

```

4.7 Wallet Service

Full design is covered in Section 9. The Wallet Service is the most critical service in the system and runs as an isolated process with its own PostgreSQL instance using strict ACID guarantees and double-entry bookkeeping.

4.8 Trust & Moderation Service

Full design is covered in Section 10.

4.9 Event Service (CampusConnect)

Responsibility: Event creation and management, ticket issuance, RSVP management, and team-finder board.

Data Model:

```

campus_events
  id                UUID PRIMARY KEY
  organiser_id      UUID REFERENCES users(id)
  campus_id         UUID
  title             VARCHAR(200)
  description        TEXT
  category
ENUM('fest','hackathon','workshop','sports','social','placement',
'other')
  start_time        TIMESTAMPTZ
  end_time          TIMESTAMPTZ
  venue             VARCHAR(200)

```

```

max_capacity      INTEGER
ticket_price      DECIMAL(8,2) DEFAULT 0
is_team_event     BOOLEAN DEFAULT FALSE
team_size_min     SMALLINT
team_size_max     SMALLINT
banner_url        TEXT
status
ENUM('draft','published','cancelled','completed')

event_registrations
  id              UUID PRIMARY KEY
  event_id        UUID REFERENCES campus_events(id)
  user_id         UUID REFERENCES users(id)
  ticket_id       VARCHAR(20) UNIQUE    -- QR code value
  team_name       VARCHAR(100)
  payment_id      UUID
  registered_at   TIMESTAMPTZ DEFAULT NOW()

```

4.10 Notification Service

Full design covered in Section 12.

4.11 Campus Admin Service

Responsibility: Campus configuration, vendor onboarding, ambassador management, analytics dashboard, and content moderation queue.

This service powers the web-based admin panel used by the Campus Operations team. It is read-heavy and connects to a read replica of the primary database and TimescaleDB for analytics.

5. Database Design & Data Architecture

5.1 Database-Per-Service Pattern

Each service has exclusive ownership of its database. Cross-service data access is done only through API calls or by subscribing to Kafka events. This enforces service boundaries and prevents the shared-database anti-pattern that makes microservices unmaintainable.

5.2 Read Replicas & CQRS

High-read services (Marketplace, Event, Notification) run with a PostgreSQL read replica. The Command Query Responsibility Segregation (CQRS) pattern is applied: writes go to the primary, reads go to the replica. For the Marketplace and SkillHub, the read model is maintained in Elasticsearch for full-text search and complex filtering.

5.3 Database Migrations

All schema changes are managed through **Flyway** (Java-based, used via Docker). Every migration is versioned, forward-only, and reviewed in pull request before deployment. Zero-downtime migrations are enforced — no locking migrations in production. The expand-contract pattern is used for column renaming and breaking changes.

5.4 Data Archival

Orders, transactions, and audit logs older than 2 years are archived to AWS S3 (Parquet format via AWS Glue) and removed from the live database. Archived data remains queryable via AWS Athena for compliance and analytics purposes.

6. API Design Standards

6.1 REST API Conventions

All REST APIs follow these conventions uniformly:

Base URL: `https://api.campusly.in/v1`
Authentication: Bearer <JWT> in Authorization header
Pagination: Cursor-based (not offset) for all list endpoints
Date Format: ISO 8601 (2026-04-23T10:00:00Z) in all request/response
Error Format: `{ "error": { "code": "LISTING_NOT_FOUND", "message": "...", "details": {} } }`
Success Format: `{ "data": { ... }, "meta": { "cursor": "...", "total": 120 } }`

Versioning Strategy: URI-based versioning (/v1/, /v2/). A new major version is introduced only for breaking changes. Old versions are deprecated with a 6-month sunset notice.

6.2 API Rate Limiting

Rate limits are enforced at the API Gateway layer using a token bucket algorithm per user (identified by JWT sub claim):

Endpoint Class	Limit
Standard API calls	100 req / minute
Search endpoints	30 req / minute
Payment initiation	10 req / minute
Authentication (login/OTP)	5 req / 15 minutes
Media upload	20 req / hour

Exceeding limits returns 429 Too Many Requests with a Retry-After header.

6.3 API Documentation

All APIs are documented using **OpenAPI 3.1** spec, committed to the repository alongside the service code. A Swagger UI instance is deployed at <https://api.campusly.in/docs> for internal use. Client SDKs for TypeScript (mobile/web) are auto-generated from the OpenAPI spec using `openapi-generator`.

7. Mobile Application Architecture

7.1 Application Structure (React Native)

```
src/
├── app/                                # Expo Router file-based routing
│   ├── (auth)/                        # Login, register, OTP screens
│   ├── (tabs)/                        # Main tab navigator
│   │   ├── home/                     # Feed and discovery
│   │   ├── marketplace/              # Buy & sell
│   │   ├── food/                     # CampusEats
│   │   ├── gigs/                     # QuickGigs + RunIt
│   │   └── profile/                  # User profile, wallet, settings
│   └── _layout.tsx
├── components/
│   ├── ui/                           # Design system components
│   ├── marketplace/                  # Listing cards, search, filters
│   ├── food/                         # Menu, cart, order tracker
│   └── shared/                       # Navigation, modals, toasts
├── services/                          # API client modules (one per
backend service)
├── stores/                            # Zustand state stores
├── hooks/                             # Custom React hooks
├── utils/                             # Formatters, validators, helpers
└── constants/                        # Theme, routes, config
```

7.2 Navigation Architecture

Bottom Tab Navigator houses the five primary modules. **Stack Navigators** handle deep flows within each module (listing detail → purchase → confirmation). **Modal Screens** are used for quick-action flows (post a listing, create a task, add to cart).

Deep Linking is implemented from launch — every listing, event, and profile has a unique URL (`campusly.in/listing/:id`) that opens directly in the app via Universal Links (iOS) and App Links (Android).

7.3 State Management

Zustand manages global client state: authentication, wallet balance, notification count, and user preferences. **React Query** manages all server state: fetching, caching, background refetching, and optimistic updates. The split is deliberate — Zustand for things that are truly client-only, React Query for anything that mirrors server state.

```
// Example: Listing store with optimistic update
const useListingStore = create<ListingStore>((set) => ({
  filters: defaultFilters,
  setFilters: (filters) => set({ filters }),
}));

// React Query handles the data fetching with 5-minute stale
time
const { data: listings } = useQuery({
  queryKey: ['listings', filters],
  queryFn: () => listingService.getListings(filters),
  staleTime: 5 * 60 * 1000,
});
```

7.4 Offline Capability

The app is designed for intermittent connectivity — a reality on large campuses with crowded networks. Critical views (wallet balance, active orders, own listings) are cached locally using **MMKV** (fast key-value storage for React Native). Write operations (place order, confirm delivery) are queued using **React Native MMKV** + a custom sync queue that retries on reconnection.

7.5 Performance Standards

The app targets the following performance benchmarks, measured on a mid-range Android device (e.g., Redmi Note 12):

- **App cold start to interactive:** < 2.5 seconds
- **Screen transition:** < 300ms
- **API response (P95):** < 500ms
- **Image load (lazy, CDN):** < 1 second
- **APK size (Android):** < 40MB

Performance monitoring is done via **Sentry Performance** and **Firebase Performance Monitoring**.

8. Real-Time Systems

8.1 WebSocket Infrastructure

Real-time features (order tracking, chat, task assignment, live notifications) are powered by **Socket.IO** running on dedicated WebSocket servers behind a sticky-session load balancer. The Socket.IO server uses Redis pub/sub as the adapter — enabling horizontal scaling across multiple WebSocket server instances without losing message delivery.

```
Client ↔ WebSocket Server (Socket.IO)
      ↓
    Redis Pub/Sub
      ↓
  [Auth, Food, Errand Services]
emit events → Redis → WebSocket server → client
```

Connection Authentication: WebSocket connections are authenticated via a short-lived WebSocket token (separate from the API JWT, issued by the Auth Service, valid for 1 hour). This prevents token reuse attacks via WebSocket.

8.2 Geospatial Matching Engine (Go)

The delivery partner and runner assignment system is a dedicated Go service using PostGIS extensions on PostgreSQL for geospatial queries.

Delivery Partner Location Updates:

- Active delivery partners and runners send their GPS coordinates every 5 seconds via WebSocket.
- Coordinates are stored in Redis as a sorted set using the GeoAdd command (`GEOADD campus:delivery:active <lon> <lat> <user_id>`).
- Redis GEORADIUS queries return partners within a specified radius in $O(N+\log M)$ time.

Assignment Algorithm:

1. On new food order / errand task:
2. Query Redis for available delivery partners within 800m of pickup
3. Sort by: $(\text{distance} \times 0.6) + (\text{rating} \times 0.4)$ score
4. Send push notification to top 3 candidates simultaneously
5. First to accept within 60 seconds is assigned
6. If no acceptance: expand radius to 1.5km, retry
7. If still no acceptance: notify requester of delay, queue for manual assignment

8.3 Chat System

In-app messaging between buyers and sellers, clients and skill providers, and task requestors and runners is built on a simple but scalable design:

- Messages are persisted in **MongoDB** (document store, flexible schema, easy to shard by conversation_id).
- Real-time delivery via Socket.IO.
- Read receipts tracked in Redis with a TTL-based approach.
- Message search is deliberately excluded from the MVP — conversation retrieval is by conversation ID only.
- Media (images in chat) upload to S3 with a pre-signed URL flow.

Data Model (MongoDB):

```
// conversations collection
{
  _id: ObjectId,
  participants: [user_id_1, user_id_2],
  context: { type: "listing" | "skill_order" | "errand",
  ref_id: "uuid" },
  last_message: { text: "...", sender_id: "...", timestamp:
Date },
```

```

    created_at: Date
}

// messages collection (sharded by conversation_id)
{
  _id: ObjectId,
  conversation_id: ObjectId,
  sender_id: "uuid",
  type: "text" | "image" | "system",
  content: "...",
  media_url: "...",
  read_by: ["user_id_2"],
  created_at: Date
}

```

9. Payment & Wallet System

The Wallet Service is the most security-sensitive component of the platform. It is designed using **double-entry bookkeeping** — the same accounting model used by banks — to ensure that money is never created or destroyed within the system; it only moves between accounts.

9.1 Ledger Design

Every wallet has a corresponding account in the ledger. Every transaction creates two ledger entries: a debit from one account and a credit to another. The sum of all ledger entries must always equal zero.

-- Wallet Service — isolated PostgreSQL instance

```

wallets
  id          UUID PRIMARY KEY
  user_id     UUID UNIQUE NOT NULL
  balance     DECIMAL(12,2) DEFAULT 0.00
  locked_balance DECIMAL(12,2) DEFAULT 0.00 -- funds in
escrow
  currency    CHAR(3) DEFAULT 'INR'
  created_at  TIMESTAMPTZ DEFAULT NOW()

ledger_entries
  id          UUID PRIMARY KEY
  wallet_id   UUID REFERENCES wallets(id)
  transaction_id UUID REFERENCES wallet_transactions(id)
  entry_type  ENUM('debit','credit')
  amount     DECIMAL(12,2) NOT NULL CHECK (amount > 0)
  balance_after DECIMAL(12,2)
  created_at  TIMESTAMPTZ DEFAULT NOW()

```

```

wallet_transactions
  id          UUID PRIMARY KEY
  reference_type
ENUM('topup','withdrawal','escrow_lock','escrow_release',
'marketplace_sale','food_order','skill_payment',
'errand_payment','platform_fee','refund')
  reference_id  UUID          -- ID of the associated
order/listing
  amount        DECIMAL(12,2)
  status
ENUM('pending','completed','failed','reversed')
  idempotency_key VARCHAR(100) UNIQUE  -- prevents duplicate
processing
  created_at    TIMESTAMPTZ DEFAULT NOW()

escrow_holds
  id          UUID PRIMARY KEY
  payer_wallet_id UUID REFERENCES wallets(id)
  payee_wallet_id UUID REFERENCES wallets(id)
  amount        DECIMAL(12,2)
  platform_fee   DECIMAL(12,2)
  reference_type VARCHAR(50)
  reference_id   UUID
  status         ENUM('held','released','refunded')
  held_at        TIMESTAMPTZ
  released_at    TIMESTAMPTZ

```

9.2 Escrow Flow

Buyer initiates purchase:

1. Lock buyer wallet: `wallet.balance -= amount;`
`wallet.locked_balance += amount`
2. Create `escrow_hold` record (status: 'held')
3. Create two ledger entries: debit from buyer wallet,
credit to escrow system wallet

Physical exchange confirmed by buyer:

4. Release escrow: `wallet.locked_balance -= amount`
5. `Seller wallet.balance += (amount - platform_fee)`
6. Platform fee credited to Campusly system wallet
7. `escrow_hold status → 'released'`
8. Emit `transaction_completed` event to Kafka (triggers
score update, notification)

Dispute raised:

4b. Escrow remains locked pending moderation decision

5b. On resolution: full release to seller OR full/partial refund to buyer

9.3 Payment Gateway Integration

Razorpay is the primary payment gateway for wallet top-ups and vendor payouts. The integration uses Razorpay's **Payment Links** and **Route** (marketplace) products.

```
// Wallet top-up initiation
async initiateTopUp(userId: string, amount: number,
idempotencyKey: string) {
  const order = await razorpay.orders.create({
    amount: amount * 100,    // paise
    currency: 'INR',
    receipt: idempotencyKey,
    notes: { user_id: userId, type: 'wallet_topup' }
  });
  // Store order in pending_topups table
  // Return order_id and key_id to client for Razorpay
  checkout
}

// Webhook handler (Razorpay → our server)
async handleWebhook(event: RazorpayWebhookEvent) {
  if (event.event === 'payment.captured') {
    // Verify signature using HMAC-SHA256
    // Credit user wallet atomically
    // Emit topup_completed event
  }
}
```

Webhook Idempotency: All webhook events are stored in a `processed_webhook_events` table. Before processing, the system checks if the `razorpay_payment_id` has been processed before — preventing double-credits if Razorpay retries delivery.

9.4 Regulatory Compliance

Campusly operates as a payment aggregator under RBI's Payment Aggregator guidelines. All transactions above ₹10,000 require basic KYC (government ID + selfie). For vendor payouts above ₹50,000/month, GSTIN and full KYC is mandatory. TDS (1% under Section 194O) is deducted automatically on applicable transactions.

10. Trust, Safety & Moderation System

10.1 Identity Verification Layer

Tier 1 (Required for all users): Institutional email OTP verification. Phone number verification via OTP. Profile photo (manual review flag for non-face images).

Tier 2 (Required for sellers, runners, delivery partners): Government ID scan (Aadhaar / Passport), processed via a third-party verification API (IDfy or Digio). Face liveness check. Bank account verification for payouts.

Tier 3 (Required for transactions above ₹10,000): Full KYC as per RBI PA guidelines.

10.2 Content Moderation Pipeline

All user-generated content — listing photos, skill profiles, event banners, chat messages — passes through a moderation pipeline:

Content submitted → Pre-moderation (automated) → Post-moderation (human review on flags)

Automated checks:

- Image: AWS Rekognition for NSFW, text in image extraction (Textract)
- Text: Keyword filter + toxicity ML model (AWS Comprehend)
- Price: Anomaly detection (statistical outlier vs. category median)

Automated decisions:

- Score ≥ 0.9 confidence safe → auto-approve
- Score ≥ 0.7 confidence violation → auto-reject + user notification
- Score in between → queue for human review (< 2 hour SLA)

Human review (Moderation Dashboard — internal web tool):

- Moderators see content, automated scores, user history
- Actions: approve / reject / warn / suspend / escalate

10.3 Fraud Detection

A rule-based fraud engine (extensible to ML) runs on every transaction event:

Rules (evaluated in real-time via streaming on Kafka):

- Same user buying and selling with themselves (multiple accounts)
- Listing price significantly below campus market median (potential scam bait)
- New account (<7 days) attempting high-value transaction (>₹5,000)
- Multiple failed OTPs from same device

- Withdrawal request within 24 hours of large deposit (money laundering signal)
- Unusual geographic IP (VPN / proxy access)

Actions on trigger:

- Low severity: log, increase monitoring
- Medium severity: require additional verification for next action
- High severity: freeze account, flag for manual review, notify user

11. Search & Discovery Engine

Elasticsearch powers all search across the platform. A single ES cluster (3-node, replicated) handles indexing and querying for Marketplace, SkillHub, Events, and Vendors.

11.1 Index Design

```
// marketplace_listings index mapping
{
  "mappings": {
    "properties": {
      "id": { "type": "keyword" },
      "title": { "type": "text", "analyzer": "standard" },
      "description": { "type": "text", "analyzer": "standard"
    },
    "category": { "type": "keyword" },
    "price": { "type": "scaled_float", "scaling_factor":
100 },
    "condition": { "type": "keyword" },
    "campus_id": { "type": "keyword" },
    "seller_score": { "type": "float" },
    "pickup_location": { "type": "keyword" },
    "created_at": { "type": "date" },
    "status": { "type": "keyword" },
    "boost_score": { "type": "float" }    // paid listing
boost multiplier
    }
  }
}
```

11.2 Search Ranking Algorithm

Listings are ranked using Elasticsearch's **function_score** query combining multiple signals:

Final Score = BM25 relevance score (text match)

- × decay function (newer listings scored higher, half-life: 3 days)
- × seller_score (Campusly Score of seller, normalised)
- × boost_multiplier (1.0 for standard, 2.5 for premium listings)

11.3 Recommendations Engine (Phase 2)

A collaborative filtering recommendation engine (Matrix Factorisation using Apache Spark MLlib) is introduced in Phase 2. It uses implicit feedback (views, saves, purchases) to generate personalised "You might like" feeds. The model is retrained weekly on the previous 30 days of interaction data.

12. Notifications Infrastructure

12.1 Push Notifications

FCM (Firebase Cloud Messaging) handles push notifications for both Android and iOS. The Notification Service maintains a device token registry and handles token refresh, invalid token cleanup, and notification delivery status tracking.

Kafka consumer group: notification-workers (5 consumers)

- Consumes events: order.status_updated, message.received, task.assigned, payment.received, listing.sold, etc.
- Each consumer fetches user preferences (DND hours, notification types)
- Renders notification template with event data
- Dispatches to FCM for push / SMS gateway for SMS / email for email
- Records delivery attempt in notifications_log table

12.2 Notification Preferences

Users can configure notification preferences per category (transactions, messages, events, promotions) and per channel (push, SMS, email). DND hours (e.g., 11 PM – 7 AM) are respected for non-urgent notifications. Critical notifications (payment received, dispute opened, account security) always deliver regardless of preferences.

12.3 In-App Notification Centre

A real-time notification bell is powered by Server-Sent Events (SSE) for the web dashboard and WebSocket events for the mobile app. Unread count is maintained in Redis (`INCR user:{id}:unread_notifications`) and decremented on read.

13. Authentication & Identity

13.1 JWT Architecture

Access Token: HS256 signed JWT, expires in 15 minutes
Payload: { sub: user_id, campus_id, role, iat, exp, jti }

Refresh Token: Opaque random string (256-bit), stored in Redis

Redis key: refresh:{token_hash} → { user_id, device_id, expires_at }

Expires: 30 days (rolling)

Token Rotation: On every refresh, old refresh token is invalidated and a new one is issued. This limits the blast radius of a stolen refresh token.

13.2 Institutional Email Verification Flow

1. User submits registration with email: student@srmist.edu.in
2. System validates domain against campus allowlist
3. 6-digit OTP generated (crypto.randomInt), stored in Redis with 10-minute TTL
4. OTP sent to institutional email via AWS SES
5. User submits OTP → verified → account created → access + refresh token issued
6. Future campuses: add their domain to the allowlist table, zero code change

13.3 Multi-Device Support

Users can be logged in on up to 5 devices simultaneously. Each device has an independent refresh token. The session management dashboard (in-app Settings) lists all active devices with last-seen timestamp and allows remote logout of any device.

14. Security Architecture

14.1 Defence-in-Depth Layers

Network Layer: All traffic is encrypted via TLS 1.3. The API Gateway is the only public-facing endpoint — all backend services are within a private VPC subnet with no public IP. AWS Security Groups enforce least-privilege network access between services.

Application Layer: SQL injection prevention via parameterised queries (never string-concatenated SQL). XSS prevention via input sanitisation (DOMPurify on web, text-only rendering on mobile). CSRF protection via SameSite=Strict cookies for web sessions. All file uploads validated for MIME type and file size before S3 ingestion.

Data Layer: Sensitive fields (bank account numbers, government ID numbers) are encrypted at rest using AES-256 with keys managed in AWS KMS. PII is stored in the minimum set of services

necessary and excluded from analytics pipelines. Database credentials are rotated every 90 days via AWS Secrets Manager.

Secrets Management: All environment variables, API keys, and credentials are stored in AWS Secrets Manager and injected into containers at runtime via the AWS Secrets Store CSI driver. No secrets in environment files or Docker images.

14.2 Security Testing

Dependency Scanning: `npm audit` and Snyk run on every PR to detect vulnerable packages.

SAST (Static Application Security Testing): Semgrep runs on every PR via GitHub Actions.

Penetration Testing: An external pentest is conducted before each major version release and annually thereafter. **Bug Bounty:** A responsible disclosure programme is launched after Series A with bounties for verified CVEs.

15. DevOps, CI/CD & Infrastructure

15.1 Environment Structure

Environment	Purpose	Deployment Trigger
Development	Local development	Manual / docker-compose
Staging	Integration testing, QA	Merge to <code>main</code> branch
UAT	Pre-release user acceptance	Manual promotion from staging
Production	Live environment	Manual promotion from UAT with approval gate

15.2 CI/CD Pipeline (GitHub Actions)

On every Pull Request:

- Lint (ESLint, Prettier check)
- Type check (TypeScript strict mode)
- Unit tests (Jest) – must pass with >80% coverage
- Integration tests (Supertest against test database)
- Docker build (verify image builds successfully)
- Snyk security scan
- Semgrep SAST scan
- OpenAPI spec validation
- Preview deployment (Vercel for web, Expo EAS for mobile preview build)

On merge to main (staging deploy):

- All PR checks +
- E2E tests (Playwright for web, Detox for mobile)

- Docker image build + push to AWS ECR
- Helm chart update → ArgoCD detects and deploys to staging cluster
- Smoke tests on staging
- Slack notification to #deployments channel

Production deploy:

- Manual approval in GitHub Actions (requires 2 engineering leads)
- Blue/green deployment via Kubernetes
- Automated rollback if error rate > 1% within 5 minutes of deploy

15.3 Kubernetes Configuration

The production cluster runs on **AWS EKS** in the **ap-south-1** (Mumbai) region. Multi-AZ deployment across 3 availability zones ensures high availability. Node groups are split by workload type: compute-optimised nodes for the matching engine and transaction processing, and general-purpose nodes for API services.

Key Kubernetes resources per service:

Each service has:

- Deployment (with rolling update strategy, maxSurge: 1, maxUnavailable: 0)
- Service (ClusterIP for internal, LoadBalancer via AWS ALB for public-facing)
- HorizontalPodAutoscaler (CPU: scale at 70%, Memory: scale at 75%)
- PodDisruptionBudget (always keep at least 50% of pods available)
- ConfigMap (non-secret configuration)
- ExternalSecrets (sync secrets from AWS Secrets Manager)
- NetworkPolicy (restrict pod-to-pod communication to declared dependencies only)

15.4 GitOps with ArgoCD

ArgoCD manages all Kubernetes deployments. The desired state of the cluster is always represented by Helm charts in the **infra/** directory of the monorepo. ArgoCD continuously reconciles the cluster state to match the repo state — no manual **kubectl apply** in production.

16. Observability & Monitoring

16.1 The Three Pillars

Metrics: **Prometheus** scrapes metrics from all services. **Grafana** dashboards visualise business metrics (GMV per hour, active orders, delivery times) and technical metrics (API latency P50/P95/P99, error rates, pod CPU/memory). Alerting rules fire PagerDuty escalations for critical thresholds (error rate > 1%, P95 latency > 2s, wallet transaction failures > 0).

Logs: All services emit structured JSON logs (via **Pino** for Node.js, **zerolog** for Go) to stdout. Kubernetes collects these via Fluent Bit and ships to **AWS CloudWatch Logs** for storage and **Elasticsearch** for search and analysis. Log retention is 90 days hot (ES), 2 years cold (S3 Glacier).

Traces: **OpenTelemetry** SDK is instrumented in all services. Traces are exported to **AWS X-Ray** for distributed tracing. Every API request carries a `trace_id` that propagates through all service calls — enabling end-to-end request tracing for debugging latency issues in multi-service flows.

16.2 SLO Definitions

Service	SLO	Measurement Window
API Gateway availability	99.9%	30-day rolling
API latency P95	< 500ms	24-hour rolling
Wallet transaction success rate	99.99%	30-day rolling
Push notification delivery	95% within 30s	24-hour rolling
Food order delivery accuracy	98% on-time within estimated window	7-day rolling

Error budgets are calculated from SLOs and tracked in a Grafana dashboard. When error budget consumption rate exceeds 50% of the monthly budget in a week, a feature freeze is declared and the engineering team focuses exclusively on reliability.

17. Phased Implementation Roadmap

Phase 0 — Foundation (Weeks 1–8)

The engineering goal of Phase 0 is to have a deployable, tested MVP ready for the SRM pilot launch. This phase prioritises breadth over depth — a thin slice of every module is built to production quality.

Week 1–2: Set up monorepo (Turborepo), CI/CD pipeline, AWS infrastructure (VPC, EKS cluster, RDS PostgreSQL, ElastiCache Redis, S3 bucket), and establish coding standards, branch strategy, and PR review process. Set up Auth Service and User Service with full test coverage.

Week 3–4: Build Marketplace Service (listing CRUD, image upload, basic search). Build CampusWallet (top-up via Razorpay, peer transfer, ledger). Integrate payment gateway webhooks with idempotency handling.

Week 5–6: Build React Native mobile app shell — tab navigation, auth flows, home feed. Implement Marketplace screens: listing browse, listing detail, create listing, purchase flow with escrow. Build Notification Service (FCM integration, basic push notifications).

Week 7–8: Build QuickGigs and RunIt (task board, task creation, basic matching). End-to-end testing across all Phase 0 services. Penetration test by a team member with security background. Bug bash and performance profiling on a target device (Redmi Note 12).

Phase 0 Exit Criteria: 50 test users can register, list items, buy items with escrow, post tasks, and receive payments — all on the production environment without errors.

Phase 1 — Pilot Launch (Weeks 9–20)

Week 9–12: Launch CampusEats — vendor onboarding flow (admin panel), menu management, order placement, delivery partner app (same app, partner mode toggle), real-time order tracking via WebSocket. Launch SkillHub — skill profile creation, service listing, order flow, milestone-based escrow.

Week 13–16: Launch CampusConnect — event creation, RSVP, team finder, collaboration boards. Implement in-app chat (Socket.IO + MongoDB). Implement Campusly Score computation (event sourcing + nightly job). Launch admin moderation dashboard.

Week 17–20: Performance optimisation based on real usage data. Elasticsearch search quality tuning based on query analytics. A/B test notification copy and push notification timing. Implement referral programme mechanics. Conduct user interviews with 20 active users and ship a prioritised patch release.

Phase 2 — Multi-Campus Expansion (Months 6–18)

Architecture changes for multi-tenancy: Campus is a first-class entity in every data model. The schema supports multiple campuses with campus-specific configuration (allowlisted email domains, local vendor onboarding, geo-boundaries for delivery). A Campus Config Service manages per-campus feature flags — allowing selective feature rollout per campus.

New engineering capabilities added in Phase 2: Recommendation engine (collaborative filtering on interaction data), advanced fraud detection ML model, vendor analytics dashboard, self-serve vendor onboarding portal, affiliate/ambassador attribution system, B2B insights API for brand partners.

Infrastructure changes for scale: Move from single-region to multi-AZ with active-active configuration. Implement read replicas for all high-traffic services. Introduce Kafka Schema Registry to enforce event schema contracts as producer/consumer teams diverge. Implement a distributed rate limiting system (Redis cluster) to handle cross-campus traffic.

Phase 3 — Corporate & Financial Super-App (Months 18–36)

Architecture changes for corporate campus product: New `organisations` entity in the data model (distinct from `campuses`). Organisation-level admin roles and access controls. Corporate marketplace with invoicing support (B2B transaction requirements). Integration with corporate HRMS for employee identity verification (SAP SuccessFactors, Workday via SAML).

Financial products engineering: Partner NBFC integration API for micro-credit. Credit scoring model (uses Campusly Score + transaction history + wallet behaviour as features). Loan origination and repayment tracking. RBI compliance reporting for PA/PG regulations.

18. Team Structure & Hiring Plan

Phase 0 (Founding Team — 4–6 people)

The founding engineering team should be capable of building and deploying the Phase 0 MVP independently. Roles:

Full-Stack Lead (1): Designs and owns the backend architecture, Auth and User services, and Wallet service. Must have Node.js/TypeScript expertise and PostgreSQL experience. 3+ years.

Backend Engineers (2): Own Marketplace and Food services respectively. Node.js/TypeScript. 1–3 years. Can be SRM final-year students or fresh graduates with strong fundamentals.

Mobile Engineer (1): Owns the React Native application. Experience with React Native, Expo, and Zustand/React Query. Must have shipped at least one app to the Play Store.

DevOps / Platform Engineer (1): Sets up and owns the AWS infrastructure, CI/CD pipeline, and Kubernetes cluster. Experience with Terraform, Docker, and EKS. Can be part-time initially.

Phase 1 Additions (Months 3–6, add 4–6 engineers)

Two additional backend engineers (Food and Gigs services), one Go engineer for the matching engine, one data engineer (ETL pipelines, analytics), one QA automation engineer (Playwright + Detox), and a second mobile engineer.

Phase 2 Additions (Months 6–18, build to 20–30 engineers)

Dedicated teams form around product domains: Marketplace team, Commerce team (Eats + Errands), Payments team, Platform team (infrastructure, security, reliability), Growth Engineering team (referral, notifications, recommendations), and a Data team (analytics, ML, insights).

19. Cost Architecture

Phase 0 Monthly AWS Costs (Estimate)

Service	Spec	Monthly Cost (₹)
EKS Cluster (3 t3.medium nodes)	12 vCPU, 12 GB RAM	₹12,000
RDS PostgreSQL (db.t3.medium, Multi-AZ)	2 vCPU, 4 GB, 100 GB SSD	₹8,000
ElastiCache Redis (cache.t3.micro)	0.5 vCPU, 0.5 GB	₹2,500

S3 + CloudFront (100 GB data, 1M requests)	Storage + CDN	₹1,500
SES (10,000 emails)	Email delivery	₹300
FCM	Free up to 10M messages	₹0
Data Transfer	Egress	₹2,000
Phase 0 Total		~₹26,000/month

At Phase 2 scale (50 campuses, ~400K active users), the infrastructure cost is estimated at ₹3–5 lakhs/month, well within the margin supported by the revenue model.

20. Disaster Recovery & Business Continuity

20.1 Recovery Objectives

Tier	Components	RTO	RPO
Critical	Wallet, Auth, Payment processing	< 15 minutes	< 1 minute
High	Marketplace, Food ordering, Chat	< 1 hour	< 5 minutes
Medium	Events, Analytics, Recommendations	< 4 hours	< 1 hour
Low	Admin dashboards, Reporting	< 24 hours	< 24 hours

20.2 Backup Strategy

Database (PostgreSQL): Automated daily snapshots via AWS RDS, retained for 30 days. Point-in-time recovery (PITR) enabled with WAL archival to S3 — enabling recovery to any second in the last 7 days for critical databases. Cross-region snapshot copy to **ap-southeast-1** (Singapore) for Wallet and Auth databases.

Object Storage (S3): S3 Versioning enabled on all buckets. S3 Replication to a secondary bucket in a different AWS region.

Kafka: Retention configured for 7 days — acts as a recovery mechanism for downstream services in event of failure.

20.3 Chaos Engineering

From Phase 2 onwards, **chaos engineering** is conducted monthly using **AWS Fault Injection Simulator**. Scenarios tested include: single AZ failure (should be automatically handled by multi-AZ setup), database primary failure and failover to replica, Kafka broker failure, and delivery partner service crash during peak load. Results are reviewed and drive infrastructure hardening.

CAMPUSLY — Engineering & Software Implementation Plan

Industry-Grade Technical Blueprint — Version 2.0

> This document defines the complete software architecture, engineering standards, infrastructure design, security framework, legal compliance systems, and phased implementation plan for Campusly at production scale. It is written to the standard of a Series A-stage engineering organisation with Indian regulatory compliance as a first-class concern.

Table of Contents

1. [Engineering Philosophy](#1-engineering-philosophy)
2. [Complete System Architecture](#2-complete-system-architecture)
3. [Tech Stack — Decisions & Rationale](#3-tech-stack--decisions--rationale)
4. [Microservices — Complete Breakdown](#4-microservices--complete-breakdown)
 - Auth Service
 - User & Reputation Service
 - Marketplace Service
 - SkillHub Service
 - Food Service (CampusEats)
 - Errand Service (RunIt)
 - **CampusRide Service** — NEW
 - **CampusHelper Service** — NEW
 - **StartupHub Service** — NEW
 - QuickGigs Service
 - Event Service
 - Wallet Service
 - Notification Service
 - Trust & Fraud Service
 - Compliance Service
 - Admin & Analytics Service
5. [Database Design & Strategy](#5-database-design--strategy)
6. [API Design Standards](#6-api-design-standards)
7. [Mobile Application Architecture](#7-mobile-application-architecture)
8. [Real-Time Systems](#8-real-time-systems)
9. [Payment, Wallet & Financial Integrity](#9-payment-wallet--financial-integrity)
10. [Fraud Detection & Grey Zone Prevention Systems](#10-fraud-detection--grey-zone-prevention-systems)
11. [Legal Compliance Engineering](#11-legal-compliance-engineering)
12. [Security Architecture](#12-security-architecture)
13. [Search, Discovery & Recommendation Engine](#13-search-discovery--recommendation-engine)
14. [Notifications Infrastructure](#14-notifications-infrastructure)
15. [DevOps, CI/CD & Infrastructure](#15-devops-cicd--infrastructure)
16. [Observability & Monitoring](#16-observability--monitoring)
17. [Phased Implementation Roadmap](#17-phased-implementation-roadmap)
18. [Team Structure & Hiring Plan](#18-team-structure--hiring-plan)
19. [Infrastructure Cost Architecture](#19-infrastructure-cost-architecture)
20. [Disaster Recovery & Business Continuity](#20-disaster-recovery--business-continuity)

1. Engineering Philosophy

Five principles govern every architectural and engineering decision at Campusly. These are not aspirational values — they are enforced through code review, system design review, and architecture decision records (ADRs) that document the reasoning behind every non-trivial technical choice.

****Trust is Infrastructure, Not a Feature.**** The identity verification system, reputation engine, and fraud detection layer are not add-ons to be built after product-market fit is established. They are core infrastructure, designed and built before the first user is onboarded. Every other system depends on them, and every other system emits events that they consume.

****Financial Correctness is Absolute.**** The wallet and ledger system tolerates no inconsistency. Every other system in the platform can operate under eventual consistency. The financial layer operates under strict ACID guarantees with double-entry bookkeeping, idempotent operations, and reconciliation runs. A 1-rupee discrepancy in the ledger is a Severity-1 incident.

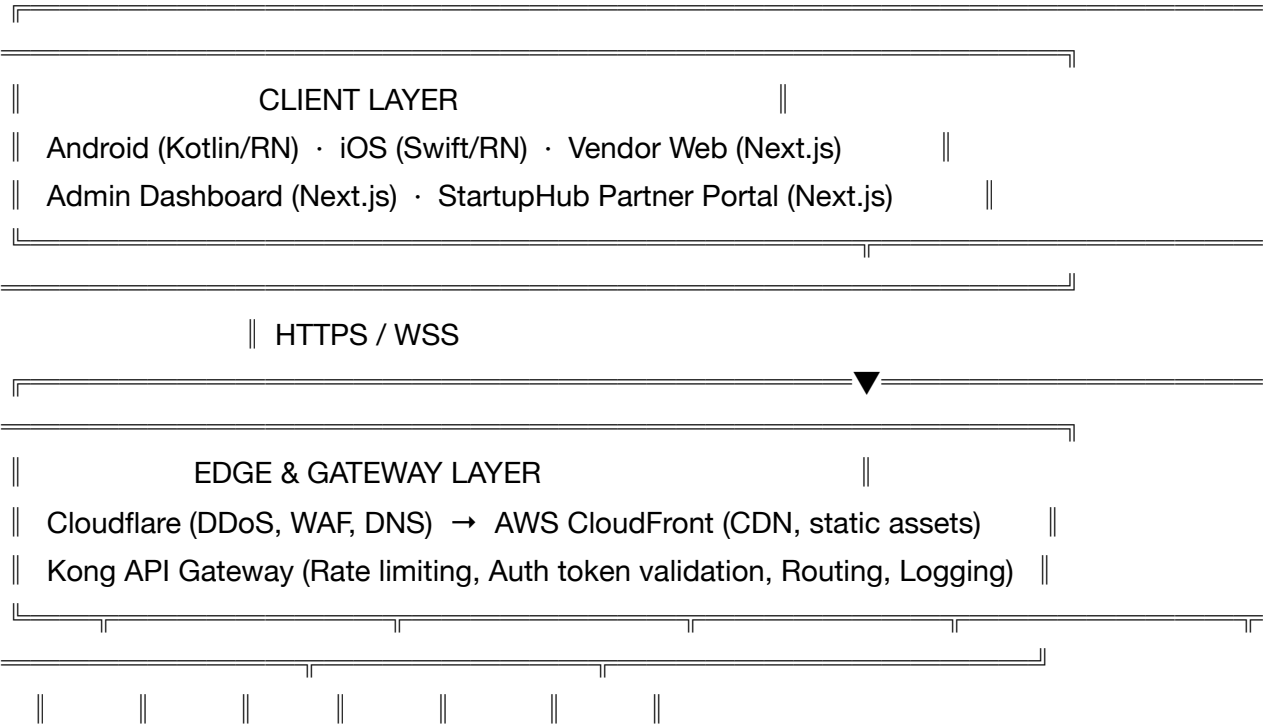
****Compliance is Architecture, Not a Checkbox.**** RBI PA guidelines, the DPDP Act, TDS obligations, transport aggregator licensing, and consumer protection rules are not dealt with by a lawyer after the product is built. They are translated into system requirements and built into the platform from Day 1. The Compliance Service is a first-class microservice with its own data store and API.

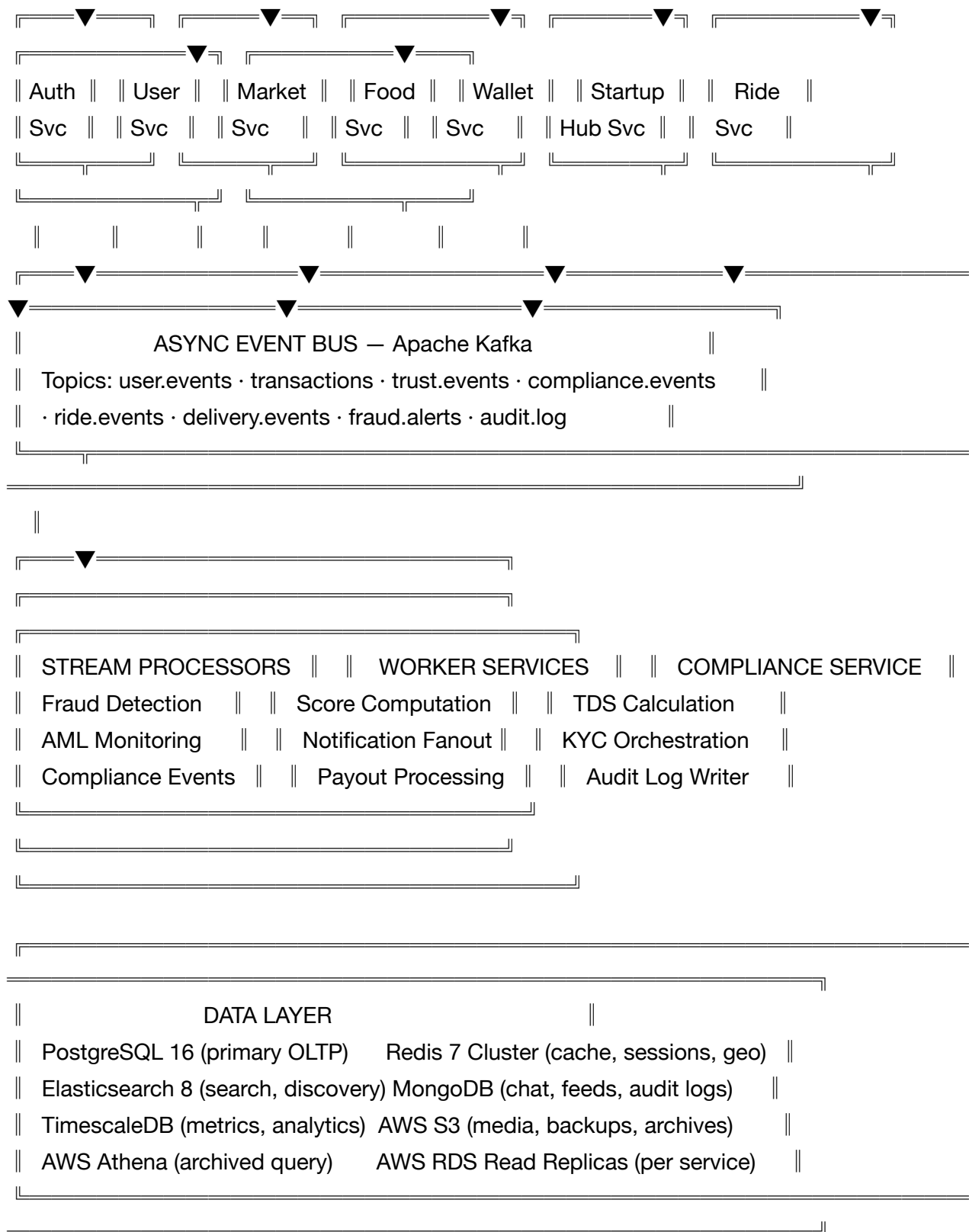
****Monorepo First, Services When Justified.**** The system is built as a well-structured modular monolith within a monorepo, with clean service boundaries honoured in code from Day 1. Services are extracted to independent deployments when scale data justifies it — not when an engineer reads a blog post about microservices. Premature distribution is a tax on velocity.

****Mobile-First, API-First.**** Every feature is defined by its API contract before any UI is built. The mobile application is a first-class consumer of the API, not a privileged client with special backend access. This discipline ensures the API is genuinely usable and enables future integrations (campus kiosks, web dashboard, third-party StartupHub integrations) without architectural rework.

2. Complete System Architecture

...





...

3. Tech Stack — Decisions & Rationale

Backend

| Layer | Technology | Rationale |

Layer	Technology	Rationale
Primary Language	Node.js (TypeScript)	Maximum velocity for API-heavy services; enormous ecosystem; excellent developer pool in SRM alumni community
High-Performance Services	Go (Golang)	Matching engine, wallet transactions, fraud stream processor — where latency and concurrency are critical
API Framework	NestJS (Node) + Gin (Go)	NestJS enforces dependency injection and clean architecture; Gin is minimal and performant
API Protocol	REST (external) + gRPC (internal)	REST for all client-facing APIs; gRPC for service-to-service for type safety and performance
Message Broker	Apache Kafka	Durable, ordered, replayable event streaming; essential for financial audit trails and fraud detection
Background Jobs	BullMQ (Redis-backed)	Payout processing, score recomputation, notification scheduling
Stream Processing	Apache Flink (Phase 2)	Real-time fraud detection on transaction streams at scale

Frontend & Mobile

Layer	Technology	Rationale
Mobile	React Native (Expo, initial)	Single codebase for Android + iOS; large talent pool; Expo's managed workflow dramatically reduces DevOps overhead in Year 1
Mobile State	Zustand + React Query	Zustand for local client state; React Query for server state caching, background refetch, optimistic updates
Admin & Partner Web	Next.js 14 (App Router)	SSR for SEO, RSC for performance; TypeScript throughout
Design System	NativeWind + Tailwind CSS	Shared design tokens across mobile and web; consistent component library

Data & Infrastructure

Layer	Technology	Rationale
Primary DB	PostgreSQL 16	ACID compliance for financial data; PostGIS extension for geospatial queries (CampusRide, delivery matching)
Cache + Real-Time	Redis 7 Cluster	Session store, rate limiting, geospatial delivery partner tracking (GEOADD/GEORADIUS), pub/sub
Search	Elasticsearch 8	Full-text search, geo-filtered listings, faceted discovery across all marketplace categories
Chat + Feeds	MongoDB	Document model fits chat messages and activity feeds; shards cleanly on conversation_id
Time-Series Analytics	TimescaleDB	GMV tracking, delivery metrics, campus-level commerce analytics
Object Storage	AWS S3 + CloudFront	Profile photos, listing images, vendor menus, compliance documents
Cloud	AWS (ap-south-1 Mumbai)	Indian data residency for DPDP compliance; lowest latency for Indian users; mature managed services
Containers	Docker + Kubernetes (EKS)	Industry standard; managed Kubernetes via AWS EKS reduces operational overhead
Infrastructure	Terraform	All infrastructure defined as code; reproducible across dev, staging, production
Service Mesh	Istio	mTLS between all services; traffic management; distributed tracing sidecar
Secret Management	AWS Secrets Manager	No secrets in environment files or container images

4. Microservices — Complete Breakdown

4.1 Auth Service

****Responsibility:**** Registration, login, OTP verification, institutional email validation, JWT issuance, token rotation, multi-device session management, and OAuth integration with university SSO systems.

****Institutional Email Validation:****

```
``typescript
// Campus domain registry — configurable per campus, no code change required
const CAMPUS_EMAIL_DOMAINS: Record = {
  'srmist.edu.in': { campus_id: 'srm-ktr', display_name: 'SRM KTR', mfa_required: false },
  'vit.ac.in': { campus_id: 'vit-vel', display_name: 'VIT Vellore', mfa_required: false },
  // Extensible to 100+ campuses without service redeployment
};

async validateInstitutionalEmail(email: string): Promise {
  const domain = email.split('@')[1];
  const config = CAMPUS_EMAIL_DOMAINS[domain];
  if (!config) throw new UnauthorizedException('Email domain not associated with a registered campus');
  return config;
}
```

****JWT Architecture:****

- Access Token: HS256 signed, 15-minute TTL, payload `{ sub, campus\_id, role, device\_id, jti }`
- Refresh Token: 256-bit random opaque string, stored in Redis with 30-day rolling TTL, bound to device fingerprint
- Token Rotation: New refresh token issued on every use; previous token immediately invalidated, preventing replay attacks
- Refresh Token Family Tracking: If a rotated (already-used) refresh token is presented, the entire family is revoked — detecting token theft

****Data Store:**** PostgreSQL (user credentials), Redis (OTP cache with 10-minute TTL, session store, refresh token families)

4.2 User & Reputation Service

****Responsibility:**** User profiles, Campusly Score computation and event sourcing, verification tier management, social graph (follow, block, report), and user search.

****Campusly Score Architecture:****

The score is event-sourced. Every user action that affects trustworthiness emits a structured `ScoreEvent` to Kafka. A dedicated Score Worker consumes these events and maintains a running score, with a full nightly recomputation as a safety net.

```
``typescript
// ScoreEvent schema (Kafka topic: trust.score_events)
interface ScoreEvent {
  user_id: string;
  event_type: ScoreEventType; // 'transaction_completed' | 'review_received' |
                               // 'dispute_raised' | 'dispute_resolved_against' |
                               // 'response_rate_tick' | 'id_verified'
  delta: number; // Positive or negative score delta
  weight: number; // Event weight in the composite formula
  metadata: Record;
  occurred_at: string; // ISO 8601
}
```

```

}

// Composite Score Formula (recalculated nightly)
// Score ∈ [0.0, 5.0]
score = (
  0.35 × transaction_completion_rate + // % of accepted transactions completed
  0.25 × avg_review_rating_normalised + // avg rating ÷ 5
  0.20 × response_rate + // % messages replied within 2 hours
  0.10 × profile_completeness + // 0–1 based on fields filled
  0.10 × inverse_dispute_rate // (1 - disputes_raised_against / total_tx)
) × 5.0
'''

**Verification Tiers:**
'''sql
user_verification_tiers
  user_id      UUID PRIMARY KEY
  tier          SMALLINT DEFAULT 0 -- 0: email only, 1: phone added, 2: govt ID, 3: full KYC
  email_verified BOOLEAN DEFAULT FALSE
  phone_verified BOOLEAN DEFAULT FALSE
  govt_id_verified BOOLEAN DEFAULT FALSE
  face_verified BOOLEAN DEFAULT FALSE
  bank_verified BOOLEAN DEFAULT FALSE
  kyc_provider_ref VARCHAR(100) -- IDfy/Digio reference ID
  verified_at   TIMESTAMPTZ
'''

**Transaction limits by tier:**
- Tier 0 (email only): Browse only, no transactions
- Tier 1 (email + phone): Transactions up to ₹500/day
- Tier 2 (+ govt ID): Transactions up to ₹10,000/day, listing items up to ₹20,000
- Tier 3 (full KYC): Unlimited, payout enabled, StartupHub vendor status available

---

### 4.3 Marketplace Service

**Key Tables:**
'''sql
listings
  id, seller_id, campus_id, title, description, price, category,
  condition, images TEXT[], pickup_point, status, boost_expires_at,
  view_count, save_count, created_at, updated_at

listing_transactions
  id, listing_id, buyer_id, seller_id, amount, platform_fee_rate,
  platform_fee_amount, escrow_hold_id, status, dispute_id,
  buyer_confirmed_at, completed_at, created_at

listing_saves      -- wishlist / saved items
listing_views      -- for search ranking signals
listing_reports    -- user-reported problematic listings
'''

**Price Anomaly Detection (runs on listing creation):**
'''typescript
async checkPriceAnomaly(category: string, price: number, campus_id: string): Promise {
  const { p25, p75 } = await this.getMarketPricePercentiles(category, campus_id);
  const iqr = p75 - p25;
  const lowerFence = p25 - 2.0 * iqr;

```

```

const upperFence = p75 + 2.5 * iqr;

if (price < lowerFence) {
  // Suspiciously cheap — potential scam bait or undervalued
  await this.trustService.flagForReview(listing_id, 'PRICE_ANOMALY_LOW', { price,
lowerFence });
}
if (price > upperFence) {
  // Suspiciously expensive — potential fraud or mistake
  await this.trustService.flagForReview(listing_id, 'PRICE_ANOMALY_HIGH', { price,
upperFence });
}
}
}

```

****Semester-End Sale Feature (Scheduled Job):****

At the start of each semester transition period (configurable per campus via admin panel), a job runs that: tags all active listings by graduating/final-year students with a `semester\_end\_sale` flag, creates a dedicated surfacing section on the home feed, and sends a push notification to all incoming-year students for the relevant campus.

4.4 SkillHub Service

Handles skill profile management, service listing, client-provider order lifecycle, milestone-based escrow triggers, and collaboration board.

****Milestone-Based Escrow for Orders above ₹1,000:****

```

```sql
skill_order_milestones
 id UUID PRIMARY KEY
 order_id UUID REFERENCES skill_orders(id)
 title VARCHAR(200)
 description TEXT
 amount DECIMAL(10,2) -- Portion of total order value
 due_date DATE
 status ENUM('pending','submitted','approved','disputed')
 submission_url TEXT -- Deliverable link (Drive, GitHub, etc.)
 submitted_at TIMESTAMPTZ
 approved_at TIMESTAMPTZ
```

```

Each milestone approval triggers a partial escrow release to the provider — protecting clients from non-delivery while protecting providers from non-payment after work is done.

4.5 Food Service (CampusEats)

****Vendor Onboarding Requirements (Enforced in the onboarding flow):****

- FSSAI licence number (validated via FSSAI API)
- Proprietor/owner Aadhaar verification
- Physical stall verification by Campus Operations team
- Menu photo upload with pricing
- Bank account for settlements (penny drop verification)

****Pre-Order Queue System:****

A core differentiator. Students order 10–30 minutes before arrival and receive a QR code. The vendor's tablet-based order management interface shows incoming pre-orders in chronological

order with preparation status (queued, preparing, ready). A physical "Campusly Pre-Order" counter at each onboarded vendor allows QR-scan pickup, bypassing the queue.

****Order State Machine:****

PLACED → CONFIRMED (vendor accepts, triggers prep timer)
→ PREPARING (vendor marks; delivery partner notified)
→ READY (for delivery orders: partner picks up / for pre-order: customer collects)
→ PICKED_UP (delivery partner confirms via QR scan at vendor)
→ DELIVERED (customer confirms via in-app or auto-confirmed after 10 min)
→ PAYMENT_RELEASED (escrow to vendor minus commission, ₹ to delivery partner)

Branch: PLACED → CANCELLED_BY_VENDOR (refund to wallet within 30 seconds)

PLACED → CANCELLED_BY_CUSTOMER (before CONFIRMED: full refund;
after CONFIRMED: no refund if prep started)

****Delivery Partner Earnings:****

₹18 base per delivery + ₹3 per 200m beyond 500m + in-app tip (capped at ₹50 to prevent tip pressure). Earnings visible in real-time in the partner dashboard. Weekly payout every Friday.

4.6 Errand Service (RunIt)

****Task Completion Verification:****

Completion is verified via a one-time PIN (6 digits) generated at task creation, displayed only to the requestor, and entered by the runner on the task completion screen. This prevents false completion claims — a runner cannot mark a task done without the physical PIN from the requestor, proving the handoff occurred.

****Runner Eligibility:****

- Tier 2 or above verification (government ID verified)
- Campusly Score ≥ 3.5 (not available to new accounts to prevent fraud)
- Explicitly opted in as a runner (separate agreement acknowledging task completion obligations)

4.7 CampusRide Service *(New)*

CampusRide handles three fundamentally different mobility models with distinct data flows and regulatory postures.

****Model A — Peer Ride Split (Dominant Model, No Transport Licence Required):****

```sql

```
ride_share_posts
 id UUID PRIMARY KEY
 poster_id UUID REFERENCES users(id)
 campus_id UUID
 origin VARCHAR(200)
 destination VARCHAR(200)
 origin_coords POINT (PostGIS)
 destination_coords POINT (PostGIS)
 departure_time TIMESTAMPTZ
 total_fare DECIMAL(8,2) -- Posted by the organiser
 max_passengers SMALLINT DEFAULT 3
 current_passengers SMALLINT DEFAULT 0
 vehicle_type ENUM('auto','cab','car','bus')
```

```

status ENUM('open','full','departed','cancelled')
created_at TIMESTAMPTZ DEFAULT NOW()

ride_share_participants
id UUID PRIMARY KEY
post_id UUID REFERENCES ride_share_posts(id)
user_id UUID REFERENCES users(id)
fare_share DECIMAL(8,2) -- Computed: total_fare / participants
wallet_deducted BOOLEAN DEFAULT FALSE
joined_at TIMESTAMPTZ

```

Fare split is computed automatically when the organiser marks the ride as departed. Each participant's fare share is deducted from their wallet and credited to the organiser, minus a 8% platform fee. The wallet deduction is atomic — all participants must have sufficient balance, or the split fails and they are notified to top up.

**\*\*Model B — Registered Driver Bookings (Requires State Aggregator Licence):\*\***

```

sql
registered_drivers
id UUID PRIMARY KEY
user_id UUID REFERENCES users(id)
driver_licence_no VARCHAR(20) UNIQUE
licence_verified BOOLEAN DEFAULT FALSE
vehicle_reg_no VARCHAR(15) UNIQUE
vehicle_type ENUM('auto','sedan','suv')
insurance_expiry DATE
puc_expiry DATE
status ENUM('pending_verification','active','suspended')
verified_at TIMESTAMPTZ
verification_doc_urls TEXT[] -- Stored in S3

driver_availability
driver_id UUID REFERENCES registered_drivers(id)
is_online BOOLEAN DEFAULT FALSE
current_location POINT (PostGIS)
last_seen TIMESTAMPTZ
-- Updated via WebSocket every 5 seconds when online

```

Driver onboarding is manual for Phase 1 — a Campus Operations team member physically verifies documents. The licence and registration are cross-checked against the government SARATHI and VAHAN databases via their public APIs.

**\*\*Model C — Campus Shuttle Feed (Zero-Cost Intelligence Layer):\*\***

```

typescript
// Crowdsourced bus status — no regulatory exposure, pure user utility
interface ShuttleFeedEvent {
 route_id: string; // From the official SRM bus schedule (static data)
 reporter_user_id: string;
 actual_departure_time: Date;
 occupancy_level: 'empty' | 'half' | 'crowded' | 'full';
 delay_minutes: number;
 comment?: string;
 reported_at: Date;
}
// Aggregated per route, displayed as real-time crowdsourced schedule
// Official schedule is baseline; crowdsourced events are overlaid

```

...

#### **\*\*Safety Features for CampusRide:\*\***

- Live ride sharing: When a participant joins a ride, they can share a live tracking link with a trusted contact (parent, friend) that shows the vehicle's GPS location in real time for the duration of the ride
- Emergency SOS: Single-tap SOS button during any active ride that simultaneously sends the GPS location to the participant's emergency contacts and the Campusly safety team
- Women's Safety Preference: Female users can filter ride-share posts to see only rides organised by other verified female students
- Incident Reporting: Any participant can report a ride incident through the app with a structured form; all incidents are logged in the Trust Service and reviewed within 2 hours

---

### ### 4.8 CampusHelper Service \*(New)\*

#### **\*\*Session Booking Flow:\*\***

...

Provider creates session type (category, price, duration, location)

- Sets weekly availability calendar
- Student browses, selects time slot, pays via wallet (escrow held)
- Booking confirmed; calendar block sent to both parties
- At session time: provider checks in via QR code at venue (proves presence)
- Session completes; student rates and releases escrow
- Provider receives (amount - 10% fee); review visible on their profile

...

#### **\*\*Session Categories and Compliance Notes:\*\***

For academic tutoring, no special licence is required — peer tutoring is not regulated as professional tutoring in India. For fitness and wellness coaching, providers must upload their certification (ISSA, ACE, or equivalent) during onboarding; the platform does not verify medical credentials (a disclaimer is displayed to clients). For career coaching and mock interviews, providers are required to declare their employment status and company; providing career guidance as a peer is not regulated, but misrepresenting professional credentials is a platform violation.

#### **\*\*Dispute Handling for Sessions:\*\***

Unlike marketplace transactions where physical item exchange is verifiable, session quality is subjective. The dispute framework for CampusHelper is therefore structured differently: disputes must be raised within 1 hour of session end; the QR check-in log proves whether the session occurred; if the session occurred and the dispute is about quality, the platform mediates with the default being release of payment (since service was rendered), with a warning added to the provider's record if the quality concern is substantiated by review history.

---

### ### 4.9 StartupHub Service \*(New)\*

StartupHub is a platform-within-a-platform. It provides external campus startups with a storefront, payment processing, campaign tools, and analytics — all within the Campusly infrastructure.

#### **\*\*Startup Onboarding Verification:\*\***

``typescript

```
// Verification pipeline for new startup onboarding
const verificationSteps: VerificationStep[] = [
 {
```

```

 step: 'founder_identity',
 type: 'govt_id_verification',
 provider: 'IDfy',
 required: true,
 // All founders must pass Aadhaar / PAN verification
 },
 {
 step: 'campus_affiliation',
 type: 'institutional_email_check',
 required: true,
 // At least one founder must have an active campus email
 },
 {
 step: 'incubator_registration',
 type: 'document_upload',
 required: false,
 // IIDCOE or equivalent registration adds 'Incubator Verified' badge
 },
 {
 step: 'product_safety_review',
 type: 'manual_review',
 required_for: ['physical_goods', 'food', 'health'],
 sla_hours: 48,
 // Manual review by Campus Operations for physical product startups
 },
];

```

**\*\*StartupHub Storefront Data Model:\*\***

```

```sql
startup_profiles
  id          UUID PRIMARY KEY
  owner_user_id  UUID REFERENCES users(id)
  campus_id     UUID
  name          VARCHAR(150) NOT NULL
  tagline       VARCHAR(250)
  description    TEXT
  category       ENUM('food','tech','service','fashion','education',
                     'health','entertainment','other')
  logo_url      TEXT
  banner_url    TEXT
  founders       JSONB[]      -- [{user_id, role, verified: bool}]
  incubator_verified BOOLEAN DEFAULT FALSE
  campusly_verified BOOLEAN DEFAULT FALSE
  subscription_tier ENUM('free','growth','scale')
  subscription_expires_at TIMESTAMPTZ
  rating         DECIMAL(3,2)
  total_orders   INTEGER DEFAULT 0
  created_at     TIMESTAMPTZ DEFAULT NOW()

startup_products_and_services
  id          UUID PRIMARY KEY
  startup_id   UUID REFERENCES startup_profiles(id)
  type         ENUM('physical_product','digital_product','service')
  name         VARCHAR(200)
  description   TEXT
  price        DECIMAL(10,2)
  images       TEXT[]
  is_available  BOOLEAN DEFAULT TRUE
  commission_rate DECIMAL(4,2) -- Overrides default if negotiated

```

****Campaign Notification System:****

StartupHub Growth and Scale subscribers can send targeted in-app push notifications to campus students, filtered by department, hostel, year of study, or interest category inferred from platform behaviour. The campaign flow:

Startup creates campaign → Selects target audience filter →
Previews estimated reach (e.g., "~3,200 students match your filter") →
Submits for review (auto-approved if no flagged keywords; manual review if triggered) →
Scheduled send at specified time → Delivery rate reported in analytics dashboard

Campaigns are clearly labelled in the notification as "Sponsored — [Startup Name]" to maintain trust with the student audience.

****Analytics Dashboard (Per Startup):****

Metrics available in the StartupHub dashboard: storefront views (daily/weekly/monthly), product click-through rate, conversion rate (views to orders), average order value, repeat customer rate (% of buyers who ordered more than once), Campusly Score of buyer cohort (a proxy for how trustworthy their customer base is), and campaign notification open rates. These metrics are computed from Campusly's internal data and displayed in a read-only dashboard accessible to the startup's registered founders.

4.10 QuickGigs Service

****Faculty Research Board Integration:****

The Faculty Research Board is a privileged sub-section of QuickGigs, access-controlled to verified faculty accounts (staff role in the user system). Faculty post tasks with a campus-verified faculty badge on the listing, increasing student trust in the task legitimacy. Payments are processed through the same escrow system. A departmental billing option allows faculty to use departmental research budgets directly, with an invoice generated by Campusly — the platform generates a GST-compliant invoice for the university's procurement records.

4.11 Event Service (CampusConnect)

****Academic Calendar Integration:****

SRM's academic calendar (exam dates, registration windows, holiday lists) is scraped from the official SRM portal at weekly intervals and stored in a structured format. This data is used for contextual surfacing (showing textbook listings prominently in the week before semester start), notification suppression (reducing promotional notifications during exam week), and event conflict detection (warning event organisers if their proposed date conflicts with a major exam or fest).

****Ticketing QR System:****

Tickets are generated as cryptographically signed QR codes. The signing key is campus-specific and rotated per event. At the event venue, organisers use a Campusly scanner mode (available on any registered organiser's device) to scan and validate tickets. Validation is done against the event registration database with the following checks: QR signature validity, ticket status (not already scanned), event time window, and match between ticket holder's registered name and presented ID (for high-security events).

4.12 Wallet Service

Detailed in Section 9. The Wallet Service runs as a fully isolated process with its own PostgreSQL instance. No other service has direct database access to the wallet schema. All wallet operations are exposed via a strictly typed gRPC API, and every operation is idempotent by design.

4.13 Trust & Fraud Service

Detailed in Section 10. The Trust Service is a consumer of Kafka events from every other service. It maintains the Campusly Score, runs fraud detection rules, manages the moderation queue, and handles the dispute resolution workflow.

4.14 Compliance Service *(New — First-Class Service)*

****Responsibility:**** TDS computation and deduction, GST invoice generation, KYC orchestration, regulatory report generation, DPDP audit log management, and suspicious transaction reporting.

```
```sql
-- TDS tracking table
tds_records
 id UUID PRIMARY KEY
 payee_user_id UUID REFERENCES users(id)
 financial_year CHAR(7) -- '2025-26'
 cumulative_payout DECIMAL(12,2) -- Running total for the FY
 tds_deducted DECIMAL(12,2) -- Running TDS deducted
 tds_rate DECIMAL(4,3) DEFAULT 0.010 -- 1% under 1940
 is_threshold_crossed BOOLEAN DEFAULT FALSE -- true when cumulative > 5L
 last_transaction_id UUID
 updated_at TIMESTAMPTZ DEFAULT NOW()

-- GST invoice table
gst_invoices
 id UUID PRIMARY KEY
 invoice_number VARCHAR(30) UNIQUE -- Format: CAMP/2025-26/000001
 billing_entity ENUM('startup','vendor','faculty_department')
 billing_entity_id UUID
 gstin VARCHAR(15)
 period_start DATE
 period_end DATE
 taxable_amount DECIMAL(12,2)
 cgst_rate DECIMAL(4,2) DEFAULT 9.00
 sgst_rate DECIMAL(4,2) DEFAULT 9.00
 total_tax DECIMAL(12,2)
 total_amount DECIMAL(12,2)
 status ENUM('draft','issued','paid','cancelled')
 issued_at TIMESTAMPTZ

-- DPDP-compliant audit log (append-only, no deletes)
data_processing_audit_log
 id UUID PRIMARY KEY
 user_id UUID
 action VARCHAR(100) -- 'data_accessed' | 'consent_given' | 'data_deleted' | etc.
 purpose VARCHAR(200)
```

```

data_categories TEXT[] -- What types of data were involved
legal_basis VARCHAR(100) -- 'consent' | 'contract' | 'legal_obligation'
actor_service VARCHAR(50) -- Which service performed the action
actor_ip_hash VARCHAR(64) -- SHA-256 of IP, not the raw IP (privacy)
created_at TIMESTAMPTZ DEFAULT NOW()
-- This table is APPEND-ONLY. No UPDATE or DELETE operations are permitted.

```

## **\*\*Automated TDS Filing Workflow:\*\***

Every Friday, the Compliance Service runs the payout computation pipeline. For each payee: check cumulative FY payouts against the ₹5 lakh threshold. If the threshold has been crossed, compute TDS at 1%, deduct it from the payout, credit it to the TDS liability account, and generate a Form 26QE record. At quarter end, file the 26QE return via the TRACES API and generate downloadable TDS certificates for all affected payees.

---

### **### 4.15 Admin & Analytics Service**

Powers the internal admin dashboard (campus operations, moderation, analytics) and the vendor/startup partner dashboard. It connects to read replicas of all service databases and to TimescaleDB for time-series analytics. It never connects to the wallet primary database — all financial operations are performed through the Wallet Service API.

---

## **## 5. Database Design & Strategy**

### **### Database-Per-Service Ownership**

Each service owns its database exclusively. Cross-service data access is performed only through the owning service's API or by consuming events from Kafka. Shared databases are a prohibited architectural pattern — any PR that establishes a direct database connection from Service A to Service B's schema is rejected in code review.

### **### Read Replicas and CQRS**

Services with high read:write ratios (Marketplace, Event, SkillHub discovery) run with PostgreSQL read replicas. The CQRS pattern separates read and write models: commands (creates, updates) go to the primary; queries (browse, search, filter) go to the replica or Elasticsearch. This halves the load on the primary for read-heavy services.

### **### Schema Migrations**

All schema changes are managed through Flyway with a strict zero-downtime migration policy in production. Breaking changes (column removal, type changes) use the expand-contract pattern: Phase 1 adds the new column without removing the old one; Phase 2 (deployed separately, typically the next sprint) removes the old column after all consumers have migrated. Column-level locking operations that would block production traffic are never performed directly — they are routed through `pg\_repack` or equivalent online schema change tooling.

### **### Data Retention and Archival**

Live database retention limits are strictly enforced to control storage costs and query performance. Active transaction records older than 6 months are moved to a cold storage partition. Records older than 2 years are archived to S3 as Parquet files via an AWS Glue ETL job and removed from the live database. All archived data remains queryable via AWS Athena for compliance and analytics.

---

## ## 6. API Design Standards

All REST APIs are governed by a uniform contract:

---

Base URL:       https://api.campusly.in/v1  
Auth Header:     Authorization: Bearer <access\_token>  
Pagination:     Cursor-based (not offset). Response includes: { data[], meta: { cursor, has\_more } }  
Date Format:     ISO 8601 throughout (2026-04-23T10:00:00+05:30)  
Error Format:     { error: { code: string, message: string, details?: object } }  
Idempotency:     POST endpoints that create financial transactions require Idempotency-Key header  
Content-Type:    application/json for all request/response bodies

---

**\*\*Rate Limiting by Endpoint Class:\*\***

Class	Limit	Window
Standard API	100 requests	Per minute, per user
Search endpoints	30 requests	Per minute, per user
Transaction initiation	10 requests	Per minute, per user
Authentication (OTP, login)	5 requests	Per 15 minutes, per IP
Media upload	20 requests	Per hour, per user
Campaign send (StartupHub)	2 requests	Per hour, per startup

**\*\*API Versioning:\*\*** URI-based (^/v1/, ^/v2/). A new major version is only introduced for breaking changes. Old versions receive a 6-month deprecation notice with a sunset date header (Sunset: Sat, 01 Jan 2027 00:00:00 GMT) on all responses before decommissioning.

**\*\*OpenAPI 3.1 Spec:\*\*** All APIs have a machine-readable OpenAPI spec committed alongside the service code. TypeScript client SDKs for the mobile app and partner portal are auto-generated from this spec using `openapi-generator-cli`, ensuring the SDK and server contract are always in sync.

---

## ## 7. Mobile Application Architecture

### File Structure (Expo Router, file-based routing)

---

```
src/
├── app/
│ ├── (auth)/ # Login, register, verify-otp, forgot-password
│ ├── (onboarding)/ # Verification tier upgrade flows
│ ├── (tabs)/ # Main tab navigator
│ │ ├── home/ # Personalised home feed
│ │ ├── marketplace/ # Browse, list, buy
│ │ ├── services/ # SkillHub, RunIt, QuickGigs, CampusHelper
│ │ ├── food/ # CampusEats — browse, cart, track
│ │ ├── ride/ # CampusRide — post, find, book
│ │ └── events/ # CampusConnect
```

```

| | | └── startups/ # StartupHub storefront discovery
| | | └── wallet/ # Balance, transactions, earnings, payout
| | | └── chat/ # In-app messaging
| | | └── settings/ # Profile, preferences, security, help
| └── components/
| | └── ui/ # Atomic design system components
| | └── marketplace/ # ListingCard, PriceTag, ConditionBadge
| | └── food/ # MenuCard, CartSheet, OrderTracker
| | └── ride/ # RideCard, FareSplitSheet, DriverCard
| | └── startup/ # StorefrontCard, CampaignBanner
| | └── wallet/ # BalanceCard, TransactionRow, PayoutCard
└── services/ # One API client module per backend service
└── stores/ # Zustand stores (auth, wallet, cart, ride)
└── hooks/ # Custom hooks (useLocation, useWallet, useCart)
└── utils/ # Currency formatting, date helpers, validators
└── constants/ # Theme tokens, route names, API config
...

```

### ### Performance Targets (Measured on Redmi Note 12, Android 13)

```

| Metric | Target |
|---|---|
| Cold start to interactive | < 2.5 seconds |
| Screen transition animation | < 300ms |
| API response P95 | < 500ms |
| Image load (via CloudFront CDN) | < 1.2 seconds |
| App bundle size (Android APK) | < 40MB |
| JS bundle size (Hermes optimised) | < 3MB |

```

### ### Offline Capability

Critical data is persisted locally using **MMKV** (10x faster than AsyncStorage): wallet balance, active order status, own listing IDs, and delivery partner shift status. Write operations that fail due to connectivity are queued in a local retry queue and automatically re-dispatched when connectivity is restored. The UI shows an "Offline Mode" banner and disables transaction-initiation actions (but not browsing) when disconnected.

---

## ## 8. Real-Time Systems

### ### WebSocket Architecture

**Socket.IO** runs on dedicated WebSocket server pods behind a sticky load balancer (AWS ALB with session affinity). Redis Pub/Sub acts as the Socket.IO adapter — when a food order status update is emitted by the Food Service, it publishes to Redis, which broadcasts to all Socket.IO server nodes, which deliver it to the connected client regardless of which pod they are connected to.

WebSocket connections are authenticated via a short-lived WebSocket token (separate from the API JWT, issued specifically for WebSocket connections, valid for 1 hour). Clients reconnect and reissue a WebSocket token on expiry.

### ### Geospatial Delivery Matching (Go Service)

Delivery partners and runners broadcast their GPS location every 5 seconds via WebSocket when their shift is active. Locations are stored in Redis using the GeoAdd command:

```
```go
// Go service — location update handler
func (s *LocationService) UpdatePartnerLocation(partnerID string, lat, lng float64) error {
    key := fmt.Sprintf("campus:%s:delivery:active", campusID)
    return s.redis.GeoAdd(ctx, key, &redis.GeoLocation{
        Name:    partnerID,
        Longitude: lng,
        Latitude: lat,
    }).Err()
}

// Assignment query — find nearest available partners within radius
func (s *MatchingService) FindNearestPartners(campusID string, pickupLat, pickupLng float64) (
    []string, error) {
    key := fmt.Sprintf("campus:%s:delivery:active", campusID)
    return s.redis.GeoRadius(ctx, key, pickupLng, pickupLat, &redis.GeoRadiusQuery{
        Radius: 800, // metres
        Unit:   "m",
        Sort:   "ASC", // nearest first
        Count:  5,    // top 5 candidates
    }).Result()
}
```
```

#### **\*\*Assignment Flow:\*\***

1. New order/task arrives → query Redis for top 5 nearest available partners within 800m
2. Score candidates:  $\text{assignment\_score} = (1/(\text{distance\_m}+1)) \times 0.6 + (\text{campusly\_score}/5) \times 0.4$
3. Send simultaneous push notifications to top 3 candidates with a 60-second acceptance window
4. First acceptance locks the assignment (Redis SETNX for atomicity)
5. If no acceptance in 60 seconds: expand radius to 1.5km, repeat with new candidates
6. If still no acceptance after 3 rounds: notify customer of delay, escalate to manual assignment queue

#### **### In-App Chat**

Messages persist in MongoDB (sharded on `conversation\_id`). Real-time delivery via Socket.IO rooms named by `conversation\_id`. Read receipts use Redis bitmaps (SETBIT conv:{id}:read:{msg\_seq} {user\_id\_position} 1). Unread counts are maintained in Redis counters and badge-synced to the app on every WebSocket connection establishment.

---

#### **## 9. Payment, Wallet & Financial Integrity**

##### **### Double-Entry Bookkeeping Architecture**

Every financial movement creates exactly two ledger entries: one debit and one credit. The algebraic sum of all ledger entries in the system must always equal zero. Violation of this invariant is a critical bug.

```
```sql
-- Wallet Service — dedicated PostgreSQL instance, isolated VPC subnet
```

```
wallets
id          UUID PRIMARY KEY
```

```

user_id      UUID UNIQUE NOT NULL
campus_id    UUID
balance      DECIMAL(14,4) DEFAULT 0.0000 -- 4 decimal places for precision
locked_balance DECIMAL(14,4) DEFAULT 0.0000 -- Funds held in escrow
currency     CHAR(3) DEFAULT 'INR'
is_frozen    BOOLEAN DEFAULT FALSE        -- Set by fraud/compliance action
freeze_reason TEXT
created_at   TIMESTAMPTZ DEFAULT NOW()
version      BIGINT DEFAULT 1             -- Optimistic locking version counter

wallet_transactions
id           UUID PRIMARY KEY
type        ENUM('topup','withdrawal','p2p_send','p2p_receive',
                'escrow_lock','escrow_release','escrow_refund',
                'platform_fee','tds_deduction','cashback',
                'referral_credit','campaign_charge')
reference_id UUID -- Foreign key to the originating domain event
reference_type VARCHAR(50)
initiator_id UUID -- User who triggered the transaction
amount       DECIMAL(14,4) NOT NULL CHECK (amount > 0)
status       ENUM('pending','completed','failed','reversed')
idempotency_key VARCHAR(128) UNIQUE NOT NULL -- Prevents double-processing
failure_reason TEXT
created_at   TIMESTAMPTZ DEFAULT NOW()
completed_at TIMESTAMPTZ

ledger_entries
id           UUID PRIMARY KEY
wallet_id    UUID REFERENCES wallets(id)
transaction_id UUID REFERENCES wallet_transactions(id)
entry_type   ENUM('debit','credit')
amount       DECIMAL(14,4) NOT NULL CHECK (amount > 0)
balance_after DECIMAL(14,4) NOT NULL -- Snapshot of balance after this entry
created_at   TIMESTAMPTZ DEFAULT NOW()
-- NO UPDATE, NO DELETE — this is an append-only immutable ledger

escrow_holds
id           UUID PRIMARY KEY
payer_wallet_id UUID REFERENCES wallets(id)
payee_wallet_id UUID REFERENCES wallets(id)
amount       DECIMAL(14,4)
platform_fee DECIMAL(14,4)
tds_amount   DECIMAL(14,4) DEFAULT 0
reference_type VARCHAR(50)
reference_id  UUID
status       ENUM('held','released','partially_released','refunded')
held_at      TIMESTAMPTZ DEFAULT NOW()
release_deadline TIMESTAMPTZ -- Auto-release if buyer doesn't confirm
released_at   TIMESTAMPTZ

```

Wallet Balance Update — Optimistic Locking Pattern

```

``typescript
// Prevents race conditions on concurrent wallet operations
async debitWallet(walletId: string, amount: Decimal, txId: string): Promise {
  const result = await this.db.query(
    UPDATE wallets
    SET balance = balance - $1,
        version = version + 1

```

```

WHERE id = $2
  AND balance >= $1      -- Sufficient balance check (atomic)
  AND is_frozen = FALSE  -- Cannot debit frozen wallets
  AND version = $3      -- Optimistic lock check
RETURNING id, balance, version
`, [amount, walletId, expectedVersion]);

if (result.rowCount === 0) {
  throw new InsufficientBalanceError('Balance insufficient or wallet state changed concurrently');
}

// Append ledger entry immediately after successful debit
await this.appendLedgerEntry(walletId, txId, 'debit', amount);
}

```

Reconciliation

An automated reconciliation job runs every midnight:

1. Sums all wallet balances in the `wallets` table
2. Queries the Razorpay API for the total funds held in Campusly's settlement account
3. Computes the expected Campusly balance (platform fees earned minus operational withdrawals)
4. Compares actual vs expected — any discrepancy above ₹1 fires a PagerDuty Severity-1 alert
5. A monthly reconciliation report is generated and reviewed by the founding team (pre-audit) or external auditor (post-Series A)

Payout Processing — Fraud-Resistant Pipeline

...

Every Friday 8:00 PM IST:

1. [Compliance Service] Computes TDS for all payees crossing FY threshold
2. [Fraud Service] Reviews week's transactions — any flagged user is quarantined
3. [Wallet Service] Generates payout batch file: {user_id, bank_account, ifsc, amount}
4. 48-hour hold: Friday 8PM → Sunday 8PM — fraud review window
5. [Wallet Service] Submits payout batch to Razorpay Route API (Sunday 8PM)
6. Razorpay processes NEFT/IMPS transfers (Monday morning for NEFT)
7. On transfer confirmation: wallet.pending_payout decremented, transaction marked COMPLETED
8. On failure: user notified, funds returned to wallet, bank details review prompted

...

10. Fraud Detection & Grey Zone Prevention Systems

Real-Time Rule Engine (Kafka Stream Processing)

Every Kafka event from every service flows through the Fraud Detection stream processor. The processor evaluates a rule set against each event in real-time and emits fraud alerts back to Kafka, which are consumed by the Trust Service for action.

```

``typescript
// Rule definitions — evaluated on every relevant event
const fraudRules: FraudRule[] = [
  {
    id: 'MULTI_ACCOUNT_DEVICE',
    trigger: 'user.registered',
    check: async (event) => {
      const count = await deviceRegistry.countActiveUsersByDevice(event.device_fingerprint);

```

```

    return count > 1;
  },
  action: 'FLAG_FOR_REVIEW',
  severity: 'HIGH',
},
{
  id: 'VELOCITY_TRANSACTIONS',
  trigger: 'wallet.transaction_completed',
  check: async (event) => {
    const count = await transactionStore.countInWindow(event.user_id, '1 hour');
    return count > 15;
  },
  action: 'REQUIRE_CAPTCHA_NEXT_ACTION',
  severity: 'MEDIUM',
},
{
  id: 'ROUND_TRIP_WALLET',
  trigger: 'wallet.topup_completed',
  check: async (event) => {
    // Detects: topup → p2p transfer → withdrawal within 24 hours
    const outflow = await walletService.getOutflowSince(event.user_id, '-24 hours');
    return outflow.withdrawal_amount >= event.amount * 0.8;
  },
  action: 'FREEZE_WALLET_PENDING_REVIEW',
  severity: 'CRITICAL',
},
{
  id: 'REVIEW_VELOCITY',
  trigger: 'user.review_received',
  check: async (event) => {
    const reviewsToday = await reviewStore.countReceivedToday(event.recipient_id);
    return reviewsToday > 8;
  },
  action: 'HOLD_REVIEWS_PENDING_AUDIT',
  severity: 'MEDIUM',
},
{
  id: 'RIDE_GHOST_TRIP',
  trigger: 'ride.trip_completed',
  check: async (event) => {
    // Compare GPS track distance with declared route distance
    const gpsDistance = await gpsTracker.computeActualDistance(event.trip_id);
    const declaredDistance = event.route_distance_km;
    return gpsDistance < declaredDistance * 0.3; // Completed in <30% of expected distance
  },
  action: 'FLAG_FOR_REVIEW',
  severity: 'HIGH',
},
];

```

Device Fingerprinting

Every client session generates a device fingerprint composed of: device model and OS version (from the User-Agent and React Native device APIs), screen dimensions, installed font list hash (web only), network type and carrier, and a hardware identifier (Android IMEI hash, iOS identifierForVendor). The fingerprint is stored server-side and used to detect multiple accounts on the same physical device, re-registration after ban, and impossible geography (same device appearing in two locations simultaneously within an implausible time window).

Graph-Based Collusion Detection (Phase 2)

A graph database (Neo4j) is introduced in Phase 2 to detect coordinated fraud rings. Nodes are users; edges are transactions, reviews, and rides. Graph algorithms (community detection, PageRank) identify clusters of highly interconnected users with unusual transaction patterns — a signal of review rings, split-fare abuse rings, or organised fraud cells.

11. Legal Compliance Engineering

DPDP Act, 2023 — Technical Implementation

****Consent Management System:****

```
```sql
user_consent
 id UUID PRIMARY KEY
 user_id UUID REFERENCES users(id)
 purpose_code VARCHAR(50) -- 'marketing_notifications' | 'data_analytics' |
 -- 'third_party_sharing' | 'location_tracking'
 is_granted BOOLEAN
 granted_at TIMESTAMPTZ
 revoked_at TIMESTAMPTZ
 consent_version VARCHAR(10) -- Version of the privacy policy at consent time
 collection_method VARCHAR(50) -- 'in_app_prompt' | 'registration_flow'
```
```

Every data processing purpose has a corresponding consent record. The mobile app's Privacy Centre (accessible from Settings) shows the user all consents they have granted, with toggle switches to revoke any consent at any time. Consent revocation triggers an automated workflow that stops all processing for that purpose within 24 hours and logs the revocation in the audit table.

****Data Subject Rights Implementation:****

- Right to Access: A "Download My Data" button in Settings triggers an automated job that compiles all personal data Campusly holds for the user, packages it as a JSON file, and makes it available for download within 48 hours.
- Right to Erasure: A "Delete My Account" request triggers a 30-day cooling-off window (during which no further processing occurs), followed by deletion of all personal data except records required to be retained by law (financial records for 8 years, TDS records for 7 years).
- Data Portability: The "Download My Data" export includes transaction history, reviews given and received, and profile data in a structured JSON format.

RBI Compliance — PPI Framework

Campusly's wallet in Phase 1 operates as a Prepaid Payment Instrument (PPI) issued under a partner PA's licence (Razorpay PPI). The technical requirements of the PPI framework that are implemented in the system:

Maximum wallet balance of ₹1,00,000 (enforced as a hard constraint in the wallet service). For wallets with balances above ₹10,000, KYC completion is mandatory before further credits. All PPI transactions are reported to the partner PA's fraud monitoring system in real time via API. No cash loading of the wallet — all top-ups must be from a bank account or debit/credit card in the user's name.

Transport Aggregator — Technical Licence Compliance

For the CampusRide driver-booking mode, the Tamil Nadu Aggregator licence requires: a grievance officer designation with published contact details, an emergency helpline number (displayed in-app during all active rides), real-time tracking data retention for 1 year, driver trip

logs accessible to the Regional Transport Authority on request, and a fare calculation algorithm transparency disclosure (published in plain language in the app's Help section).

12. Security Architecture

Network Security

All production infrastructure runs within a private VPC with no public IP addresses on any backend service. The only public-facing endpoints are the API Gateway (via AWS ALB) and S3/CloudFront for media delivery. Security Groups enforce least-privilege network access — a service can only receive traffic from the services explicitly declared as its dependencies in the network policy. Istio's mTLS enforces mutual authentication for all inter-service communication within the cluster.

****Cloudflare**** sits in front of AWS, providing: DDoS protection at the edge, WAF rules for OWASP Top 10 patterns, bot detection and CAPTCHA integration, and rate limiting at the IP level before traffic reaches the API Gateway.

Application Security

All database queries use parameterised statements — no string concatenation SQL anywhere in the codebase. User-supplied content displayed in the mobile app is rendered as plain text or through a strict allow-list HTML sanitiser (never as raw HTML). File uploads validate MIME type at both the client (for UX feedback) and the server (for security) before S3 ingestion. A file size limit of 10MB per image and 50MB per document is enforced at the API Gateway level.

****Input validation**** is performed at the API boundary using a schema validation library (Zod for TypeScript services). Any request that fails validation is rejected with a 400 error and logged for anomaly analysis.

Secrets and Credential Management

No secrets exist in environment files, Docker images, or source code. All credentials (database passwords, API keys, Razorpay secrets, JWT signing keys) are stored in AWS Secrets Manager and injected into containers at runtime using the AWS Secrets Store CSI driver. Secrets are rotated automatically every 90 days for database credentials and every 180 days for API keys, with no manual intervention required. The rotation process is tested in the staging environment before the production rotation schedule fires.

Penetration Testing and Vulnerability Management

A SAST (Static Application Security Testing) tool (Semgrep with the OWASP ruleset) runs on every pull request via GitHub Actions. A dependency vulnerability scanner (Snyk or `npm audit`) runs on the same pipeline and fails the build on critical vulnerabilities with a CVSS score above 7.0. An external penetration test is commissioned before the public launch and annually thereafter. A responsible disclosure programme with defined bounty tiers is launched post-Series A.

13. Search, Discovery & Recommendation Engine

Elasticsearch Index Architecture

A single 3-node Elasticsearch cluster (AWS OpenSearch Serverless for managed operations) serves all discovery use cases. Separate indices per entity type (listings, skills, events, startups, vendors) with explicit mappings to avoid dynamic mapping issues.

****Listing Ranking Formula:****

```

...
final_score = BM25_text_relevance          // Text match quality
              × recency_decay(created_at, half_life=3d) // Newer listings ranked higher
              × seller_quality_boost(campusly_score)   // Reputable sellers surfaced
              × listing_boost_multiplier              // 1.0 standard, 2.5 premium paid
              × availability_factor                   // 0.0 if sold/removed, 1.0 if active
...

```

****Geo-Filtered Search for Nearby Listings:****

```

'''json
{
  "query": {
    "bool": {
      "must": { "match": { "title": "laptop" } },
      "filter": [
        { "term": { "campus_id": "srm-ktr" } },
        { "term": { "status": "active" } }
      ]
    }
  },
  "sort": [
    { "_score": "desc" },
    { "created_at": "desc" }
  ]
}
'''

```

****Auto-Suggest with Typo Tolerance:****

The search bar implements auto-suggest using Elasticsearch's `search\_as\_you\_type` field type with a 2-character minimum Levenshtein edit distance (fuzziness: "AUTO"). This handles common typos (e.g., "labtop" → "laptop", "caluculus" → "calculus" in SkillHub tutoring search) that would otherwise return zero results and frustrate users.

Recommendation Engine (Phase 2)

A ****Matrix Factorisation**** model (trained using Apache Spark MLlib on AWS EMR) processes implicit feedback signals — views, saves, purchases, search queries — to generate personalised "You might like" recommendations for the home feed. The model is retrained weekly. In the interim between retraining cycles, a collaborative filtering fallback (users similar to you bought/saved these) runs in real time using Elasticsearch's "more like this" query.

14. Notifications Infrastructure

Multi-Channel Notification Delivery

...

Kafka Event (e.g., food.order.status_updated)

↓

Notification Worker (consumes from Kafka)

↓ Fetches user notification preferences

↓ Checks DND hours (no non-critical notifications between 11PM–7AM)

↓ Renders notification template with event data

↓

| | |
|--|--|
| Channel Selection (based on preferences + urgency) | |
|--|--|

| | | |
|-----------------------|------------------------|------------------------|
| Push (FCM/APNs) | In-App (Redis) | SMS (Twilio) |
| For all mobile events | Bell icon count + feed | For payment & security |

↓

Delivery attempt logged in notifications_log
 Retry on failure (max 3 attempts, exponential backoff)

Notification Priority Tiers:

- Critical (always delivered, overrides DND): Payment received, dispute opened, account frozen, security alert, active ride SOS
- High (delivered 6AM–11PM): Order status change, task assigned, booking confirmed, payout processed
- Standard (delivered 7AM–10PM, can be batched): New listing match, SkillHub enquiry, event reminder
- Promotional (respect full preference settings, daily cap of 3): Campaign notifications, new features, platform promotions

15. DevOps, CI/CD & Infrastructure

Environment Strategy

| Environment | Trigger | Purpose |
|-------------|-----------------------------------|---|
| Dev (local) | `docker-compose up` | Individual developer iteration |
| Staging | Merge to `main` | Integration testing, QA, external reviewer access |
| UAT | Manual promote from staging | Pre-release acceptance testing |
| Production | Manual approve (2 leads required) | Live platform |

CI Pipeline (GitHub Actions — Per Pull Request)

```yaml

jobs:

quality-gates:

- ESLint + Prettier check (zero tolerance for linting errors in CI)
- TypeScript strict mode compilation (no `any` without explicit justification)
- Unit tests (Jest) — coverage gate: 80% line coverage minimum
- Integration tests (Supertest, against ephemeral PostgreSQL Docker container)
- Snyk dependency vulnerability scan — fail on CVSS ≥ 7.0
- Semgrep SAST (OWASP ruleset) — fail on any HIGH or CRITICAL finding
- OpenAPI spec validation (no breaking changes without major version bump)
- Docker image build verification

preview:

- Deploy API to ephemeral staging namespace (auto-deleted after 24h)
- Expo EAS Preview build for mobile (QR code in PR comment)
- Vercel preview deployment for web dashboard

### ### CD Pipeline (Production Deployment)

```
```yaml
```

deployment:

trigger: Manual approval (both engineering lead and founding CTO must approve)

steps:

- Docker images tagged with git SHA and pushed to AWS ECR
- Helm chart values updated with new image tags
- ArgoCD detects diff and begins rolling deployment
- Canary deployment: 10% traffic to new version for 15 minutes
- Automated canary analysis:
 - Error rate must not increase by >0.5%
 - P95 latency must not increase by >100ms
- If canary passes: promote to 100%
- If canary fails: automated rollback to previous version
- Post-deploy: smoke tests (critical path scenarios)
- Slack notification to #deployments with deployment summary and link to Grafana

```
```
```

### ### GitOps with ArgoCD

The desired state of the production cluster is always represented by Helm charts in the `infra/k8s/` directory of the monorepo. ArgoCD continuously reconciles the cluster to match the repo. No direct `kubectl apply` operations are permitted in production — all changes must go through Git. The ArgoCD UI provides a visual diff of what is deployed vs what is in the repo, and a one-click rollback capability.

---

## ## 16. Observability & Monitoring

### ### Metrics (Prometheus + Grafana)

Custom business metrics are emitted by each service alongside standard technical metrics:

```
```typescript
```

```
// Business metrics — emitted per service
```

```
const metrics = {
```

```
  // Wallet Service
```

```
  wallet_topup_total: new Counter({ name: 'wallet_topup_total', help: 'Total wallet top-ups' }),
```

```
  wallet_topup_amount: new Histogram({ name: 'wallet_topup_amount_inr', buckets:  
[100,500,1000,5000] }),
```

```
  escrow_held_amount: new Gauge({ name: 'escrow_held_total_inr', help: 'Total funds in  
escrow' }),
```

```
  // Food Service
```

```
  food_order_placed: new Counter({ name: 'food_orders_total', labelNames: ['campus_id',  
'vendor_id'] }),
```

```
  food_delivery_time: new Histogram({ name: 'food_delivery_time_minutes', buckets:  
[5,10,15,20,30,45] }),
```

```
  // Marketplace
```

```
  listing_created: new Counter({ name: 'listings_created_total', labelNames: ['category'] }),
```

```
  transaction_completed: new Counter({ name: 'marketplace_transactions_total' }),
```

```
};
```

```
```
```

**\*\*Grafana Dashboards:\*\***

- Business Overview: Real-time GMV per hour, active users, orders in flight, wallet balance distribution

- Service Health: Error rate, request rate, P50/P95/P99 latency per service

- Fraud & Trust: Fraud flags per hour, account suspensions, dispute open/close rate
- Delivery Operations: Active delivery partners, average delivery time per campus, failed assignment rate

### ### Alerting (PagerDuty — On-Call Rotation)

| Condition                              | Severity       | Escalation                           |
|----------------------------------------|----------------|--------------------------------------|
| ---                                    | ---            | ---                                  |
| Wallet reconciliation mismatch         | P1 — immediate | CTO + Head of Finance simultaneously |
| API error rate > 1% (5-min window)     | P1 — immediate | On-call engineer                     |
| Fraud alert: CRITICAL tier             | P1 — immediate | On-call + Trust & Safety lead        |
| Payment gateway webhook failure        | P2 — 15 min    | On-call engineer                     |
| P95 API latency > 2s                   | P2 — 15 min    | On-call engineer                     |
| Delivery assignment failure rate > 10% | P2 — 15 min    | On-call + Campus Ops                 |
| Error rate > 0.5%                      | P3 — 1 hour    | On-call engineer                     |

---

## ## 17. Phased Implementation Roadmap

### ### Phase 0 — Foundation (Weeks 1–10)

**\*\*Engineering Objective:\*\*** Production-deployable MVP with Marketplace, Wallet, Auth, basic QuickGigs, and the complete Trust and Compliance infrastructure. The trust and compliance systems are built first — not last.

**\*\*Week 1–2:\*\*** Monorepo setup (Turborepo), GitHub Actions CI pipeline (linting, testing, building), AWS infrastructure provisioned via Terraform (VPC, EKS cluster, RDS PostgreSQL, ElastiCache Redis, S3, CloudFront, Secrets Manager). Auth Service built and unit-tested with full institutional email OTP flow.

**\*\*Week 3–4:\*\*** User Service with Campusly Score event sourcing. Compliance Service schema and TDS computation logic. Wallet Service with double-entry ledger, escrow, and Razorpay top-up integration (including webhook idempotency handling). Reconciliation job with PagerDuty alert integration.

**\*\*Week 5–6:\*\*** Marketplace Service (CRUD, image upload, price anomaly detection). React Native app shell with Expo Router, auth screens, and wallet balance screen. Elasticsearch cluster setup with marketplace listings index.

**\*\*Week 7–8:\*\*** Notification Service (FCM integration, Kafka consumer, DND logic). Trust Service with initial fraud rule engine (6 rules). RunIt Service (task posting, PIN-based completion). Admin panel skeleton (Next.js).

**\*\*Week 9–10:\*\*** End-to-end testing across all services. Load testing with k6 (simulate 500 concurrent users). Security review by one team member with a security background. Staging environment smoke tests. Recruit 75 campus ambassadors and begin vendor onboarding conversations.

**\*\*Phase 0 Exit Criteria:\*\*** 50 internal test users complete full user journeys (register, list item, buy item with escrow, post task, complete task, receive payout) on the production environment with zero data inconsistencies and zero security incidents.

---

### ### Phase 1 — Full Platform Launch (Weeks 11–28)

**\*\*Week 11–14:\*\*** CampusEats — vendor onboarding flow, tablet order management UI, delivery partner mode, real-time order tracking via WebSocket, pre-order QR pickup flow.

**\*\*Week 15–16:\*\*** CampusRide — ride-share post/join, wallet fare split on departure, campus shuttle crowdsource feed, safety features (live share, SOS button).

**\*\*Week 17–18:\*\*** CampusHelper — session type creation, availability calendar, QR check-in verification, session-specific dispute handling.

**\*\*Week 19–20:\*\*** StartupHub — storefront creation, verification pipeline, campaign notification system, analytics dashboard, subscription billing integration.

**\*\*Week 21–22:\*\*** SkillHub — skill profile, service listing, milestone escrow, collaboration board.

**\*\*Week 23–24:\*\*** CampusConnect — event creation, ticket QR system, team finder, academic calendar integration.

**\*\*Week 25–26:\*\*** In-app chat (Socket.IO + MongoDB), DPDP Privacy Centre (consent management, data download, account deletion), full KYC flow integration (IDfy), TDS automation and certificate generation.

**\*\*Week 27–28:\*\*** Performance optimisation sprint based on real usage data. Elasticsearch ranking tuning. Push notification A/B testing. Load testing at 5,000 concurrent users. Full penetration test.

---

### ### Phase 2 — Multi-Campus and Scale (Months 7–18)

Multi-tenancy hardens (campus as a configuration, not a code change). Campus domain allow-list management in admin panel. Per-campus feature flag system (launch CampusRide at VIT before CampusEats if demand signals indicate it). Graph-based fraud detection introduced (Neo4j). Recommendation engine deployed (Spark MLlib model, weekly retraining). Kafka Schema Registry enforced for all topics. Read replicas deployed for Marketplace, Food, and Event services. TimescaleDB metrics dashboards shared with campus partner administrations.

---

### ### Phase 3 — Financial Products and Corporate (Months 18–36)

Micro-credit scoring model built on top of CampusWallet transaction history and Campusly Score. NBFC partner API integration for loan origination. Corporate campus product adapts the platform schema for non-university organisations (no institutional email required; employee ID verification instead). Alumni network read-only access mode. Mobile app split into Consumer and Partner (vendor/driver) builds for Play Store compliance.

---

## ## 18. Team Structure & Hiring Plan

### ### Founding Engineering Team (Phase 0, 4–6 people)

**\*\*Full-Stack Lead:\*\*** Owns backend architecture, Auth Service, Wallet Service, and API standards. Node.js/TypeScript expert with PostgreSQL experience. 3+ years. Ideally SRM alumni with credibility in the student community.

**\*\*Backend Engineer 1:\*\*** Owns Marketplace, SkillHub, and QuickGigs services. Node.js/TypeScript. 1–3 years. Strong SQL fundamentals.

**\*\*Backend Engineer 2:\*\*** Owns Food Service, RunIt, and Notification Service. Node.js/TypeScript. 1–3 years. Experience with real-time systems (WebSockets or Socket.IO) is valuable.

**\*\*Mobile Engineer:\*\*** Owns the React Native application end-to-end. Experience with Expo, React Navigation, and at least one shipped Play Store app. Strong understanding of mobile performance profiling.

**\*\*DevOps/Platform Engineer (Part-time initially):\*\*** Sets up and owns AWS infrastructure (Terraform), Kubernetes cluster (EKS), CI/CD pipeline (GitHub Actions + ArgoCD), and monitoring stack (Prometheus + Grafana). Can transition to full-time at Phase 1 launch.

### ### Phase 1 Additions (Months 3–6)

Go Engineer for the geospatial matching engine and Wallet Service high-throughput paths. Data Engineer for ETL pipelines, TimescaleDB, and the analytics dashboard. QA Automation Engineer (Playwright for web, Detox for mobile E2E tests). Second Mobile Engineer. Campus Operations Tech Support (not an engineer — a student-facing role managing vendor onboarding and ambassador support).

### ### Phase 2 Org Structure (Months 6–18, target 20–30 engineers)

Dedicated teams coalesce around product domains: Commerce Team (Marketplace + SkillHub + QuickGigs), Operations Team (Food + Rides + Errands), Ecosystem Team (StartupHub + CampusConnect), Platform Team (infrastructure, security, reliability, compliance), Growth Engineering Team (recommendations, notifications, referral mechanics), and Data Team (analytics, ML, insights API).

---

## ## 19. Infrastructure Cost Architecture

### ### Phase 0 Monthly AWS Cost (Estimate)

| Service                    | Configuration                 | Cost (₹/month)     |
|----------------------------|-------------------------------|--------------------|
| ---                        | ---                           | ---                |
| EKS Cluster                | 3 × t3.medium nodes           | ₹12,000            |
| RDS PostgreSQL             | db.t3.medium, Multi-AZ, 100GB | ₹8,500             |
| ElastiCache Redis          | cache.t3.micro                | ₹2,500             |
| OpenSearch (Elasticsearch) | t3.small.search, 1 node       | ₹4,000             |
| S3 + CloudFront            | 50GB storage, 500GB transfer  | ₹1,800             |
| AWS SES                    | 50,000 emails                 | ₹600               |
| Secrets Manager            | 20 secrets                    | ₹400               |
| Data Transfer   Egress     |                               | ₹2,000             |
| **Phase 0 Total**          |                               | **~₹31,800/month** |

### ### Phase 1 Monthly AWS Cost (Estimate, post-launch)

Additional services: MongoDB Atlas (M10 cluster, ₹6,000), TimescaleDB (db.t3.small, ₹3,000), FCM (free), Razorpay (commission on transactions, not fixed AWS cost), IDfy KYC API (per-verification pricing, ~₹15–₹25 per KYC). Estimated Phase 1 infrastructure: \*\*₹65,000–₹80,000/month\*\*.

At Phase 2 scale (100K+ DAUs), the infrastructure cost is estimated at \*\*₹3–5 lakhs/month\*\*, well within the revenue model's margins at that scale.

---

## ## 20. Disaster Recovery & Business Continuity

### ### Recovery Objectives by Tier

| Tier | Components | RTO | RPO |
|------|------------|-----|-----|
| ---  | ---        | --- | --- |

| Critical | Wallet, Auth, Compliance | < 15 minutes | < 30 seconds |  
| High | Marketplace, Food, CampusRide | < 1 hour | < 5 minutes |  
| Medium | SkillHub, Events, Chat, Notifications | < 4 hours | < 15 minutes |  
| Low | Analytics dashboards, Admin panel | < 24 hours | < 1 hour |

### ### Backup Architecture

PostgreSQL (all service databases): Automated daily RDS snapshots retained for 30 days. Continuous WAL archival to S3 enabling point-in-time recovery to any moment in the last 7 days. For Wallet and Auth databases specifically: cross-region snapshot replication to `ap-southeast-1` (Singapore) every 6 hours. Kafka: 7-day message retention, enabling downstream service recovery from replay. MongoDB: Daily Atlas backups retained for 30 days. S3: Versioning enabled on all buckets; cross-region replication to Singapore for media assets.

### ### Multi-AZ and Failover

EKS workloads are distributed across 3 AWS Availability Zones with Pod Anti-Affinity rules ensuring that no single AZ failure eliminates more than 33% of any service's capacity. RDS Multi-AZ with automatic failover — a primary database failure triggers automatic promotion of the standby replica within 30–60 seconds, with no manual intervention required. Redis Cluster mode distributes shards across 3 AZs with automatic shard re-election on node failure.

### ### Chaos Engineering

From Phase 2 onwards, a monthly chaos experiment is conducted using AWS Fault Injection Simulator. Scenarios include: single AZ failure, RDS primary failure and failover measurement, Redis node failure, Kafka broker failure during peak load, and gradual CPU saturation of the Wallet Service to validate the rate limiting and circuit breaker configurations. Results are reviewed in the post-experiment retrospective and drive targeted reliability improvements.

---